embarca tech | Residência Tecnológica em Sistemas Embarcados

Microcontroladores – GPIO Parte 2

Unidade 4 | Capítulo 3 – Aula Síncrona (20/01/2025)

Prof. Dr Ricardo Menezes Prates

Executores:
INSTITUTO FEDERAL Piauí
INSTITUTO FEDERAL Rio Grande do Norte
INSTITUTO FEDERAL Maranhão
INSTITUTO FEDERAL Ceará
cepedi
INSTITUTO HARDWARE BR
Coordenação: Softex
Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO
GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO

- Introdução
  - Planejamento da aula
  - Detalhamento de ementa futura do curso de capacitação
- Portas Digitais (GPIO: *General-Purppose digital Input and Output*)
  - BitDogLab – Matriz de Leds (WS2812B)
  - Pico SDK – rotinas de temporização
  - Introdução ao PIO (*Programmable Input Output*)
- Exemplos de aplicação
- Conclusões

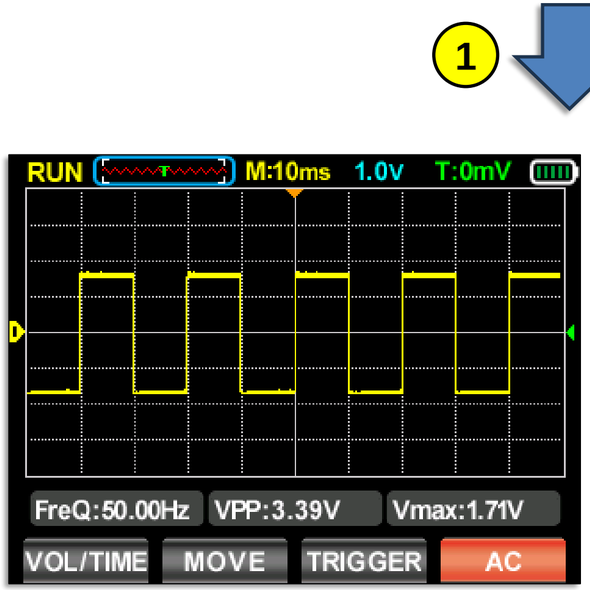
Introdução

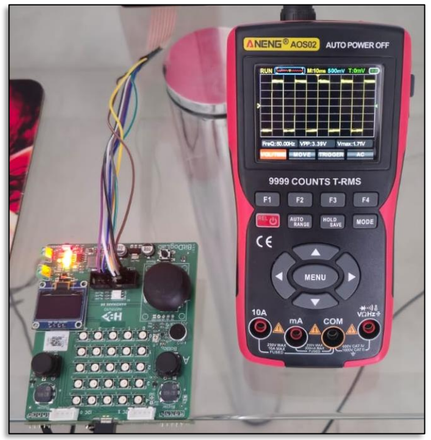
Objetivos

- Apresentar ao discente os conceitos básicos de temporização (*delay*) nos pinos **GPIO** do microcontrolador RP2040. 1

- Realizar experimentos com o **periférico interno PIO** para o controle da matriz de LEDs presente na ferramenta educacional **BitDogLab**. 2

Código fonte



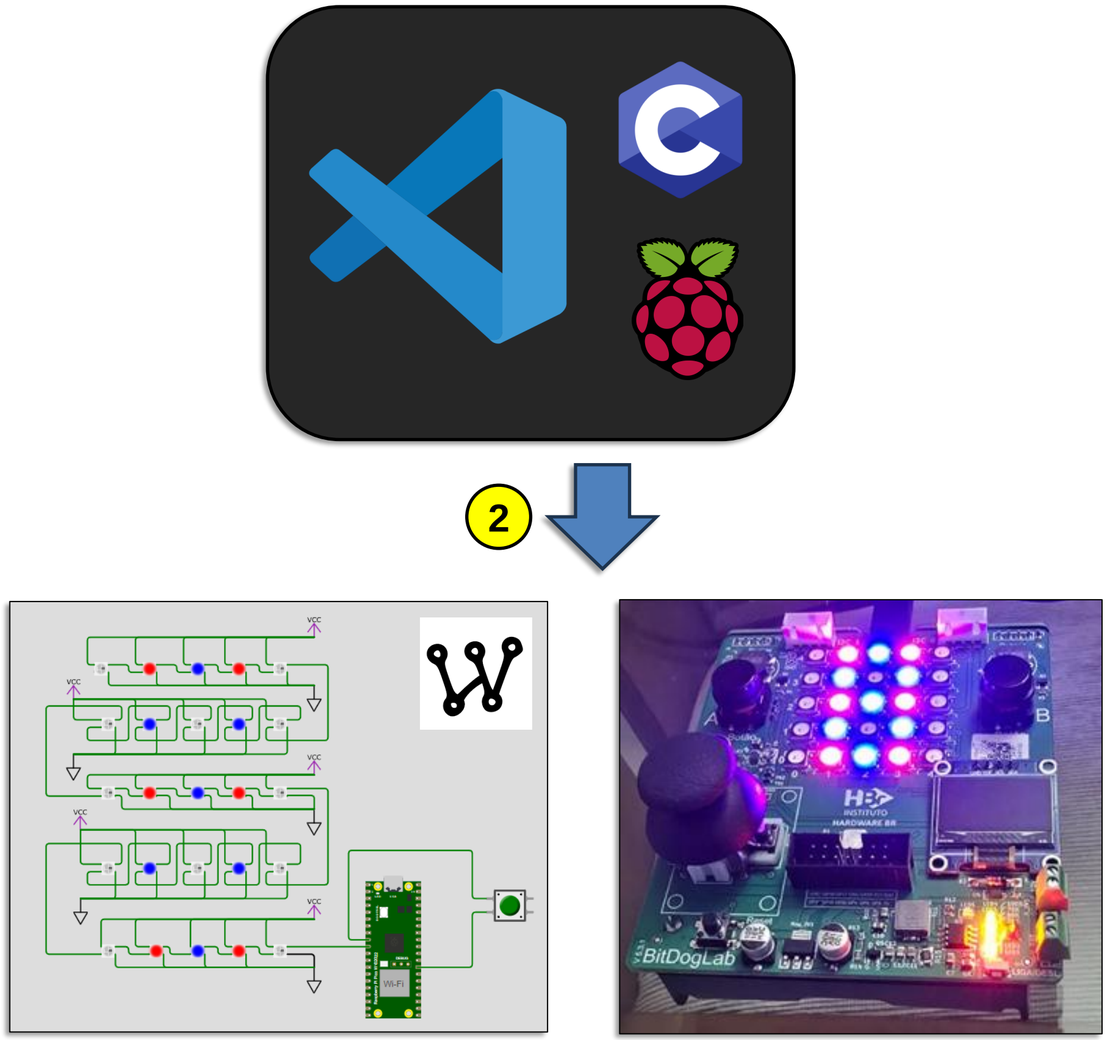


Objetivos

- Apresentar ao discente os conceitos básicos de temporização (*delay*) nos pinos **GPIO** do microcontrolador RP2040. 1

- Realizar experimentos com o **periférico interno PIO** para o controle da matriz de LEDs presente na ferramenta educacional **BitDogLab**. 2

Código fonte

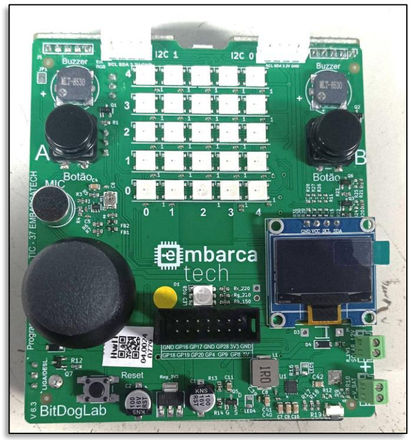


BitDogLab
V 6.3

Planejamento da Aula

ATENÇÃO: Vamos trabalhar com a combinação dos seguintes recursos:

+ + raspberrypi/**pico-sdk** + WOKWi +

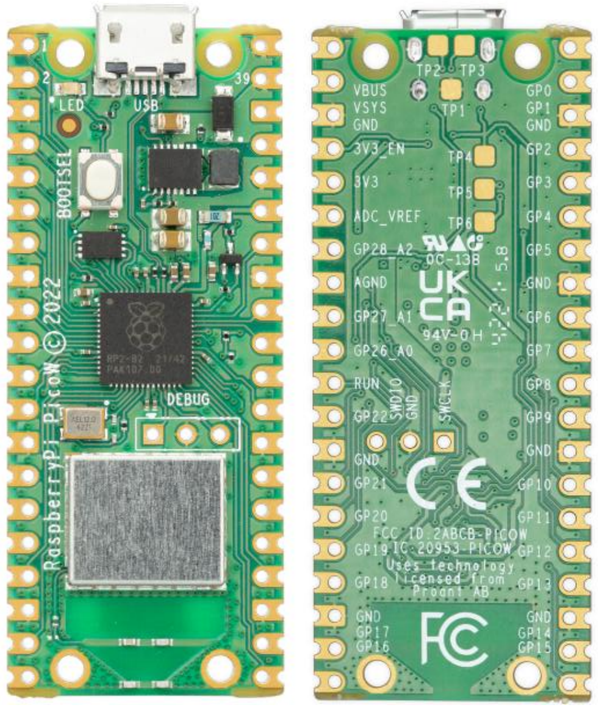


BitDogLab
V 6.3

Ementa futura do curso

- U4 - Ainda falta trabalhar com os seguintes assuntos :

1. Interrupções – 27/01;
2. *Clock* e temporizadores – 29/01;
3. Interfaces de comunicação – 03/02;
4. PWM – 05/02;
5. Conversor analógico digital – 10/02;



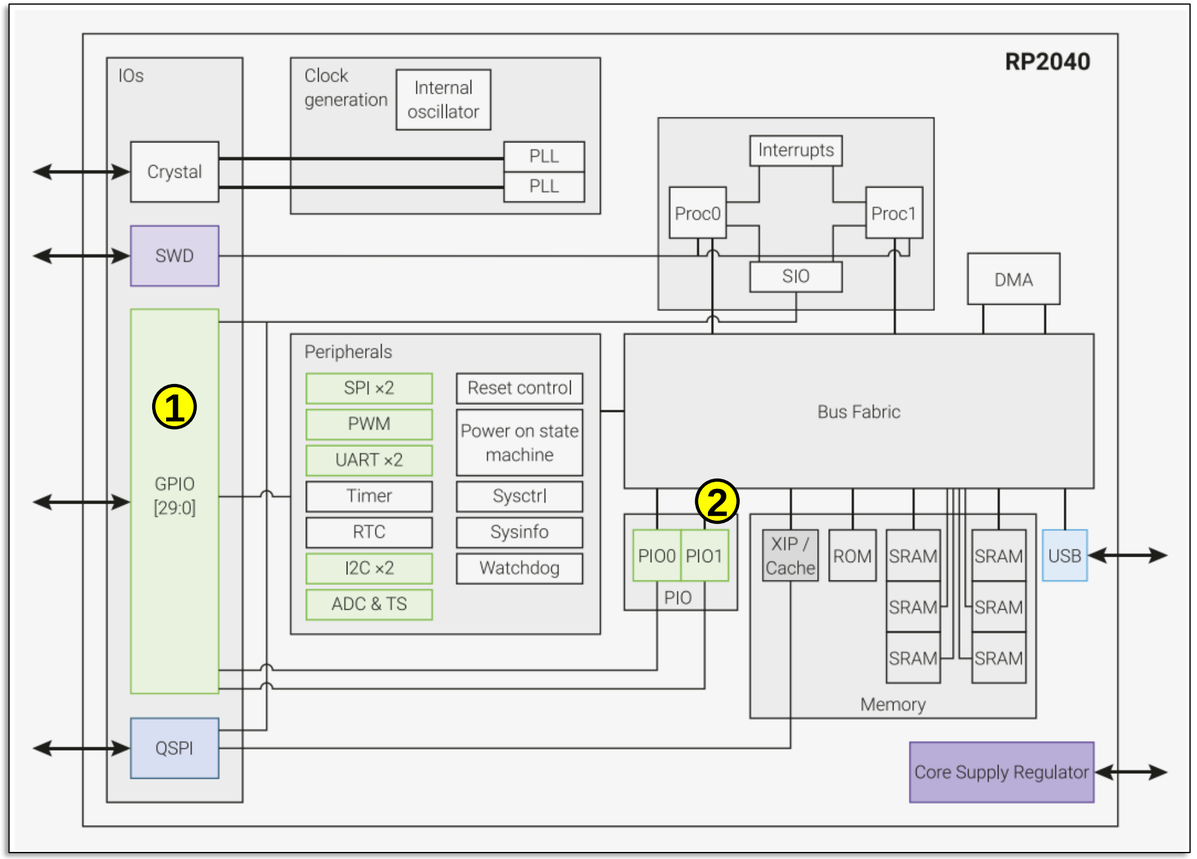
Portas Digitais – GPIO

Microcontrolador RP2040 - Revisão

- Características e funções dedicadas:

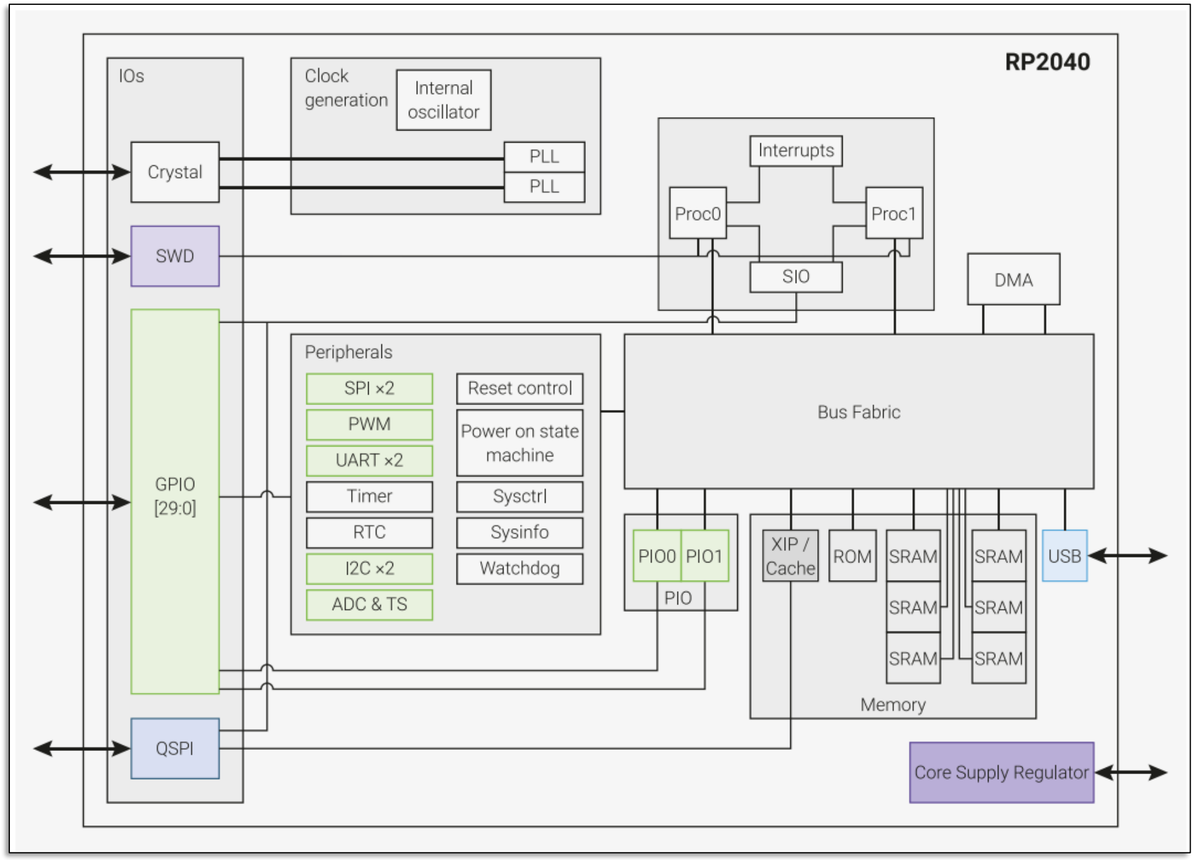
– Suporte para até 16 MB de memória Flash.
– 30 pinos GPIO. ①
– 4 entradas analógicas de 12 bits.
– 2 interfaces UART.
– 2 interfaces SPI.
– 2 interfaces I2C.
– 16 canais PWM.
– 1 controlador USB 1.1 e PHY.
– 8 máquinas de estado de E/S programável (PIO). ②
– Modo de inicialização de armazenamento em massa USB com suporte UF2.

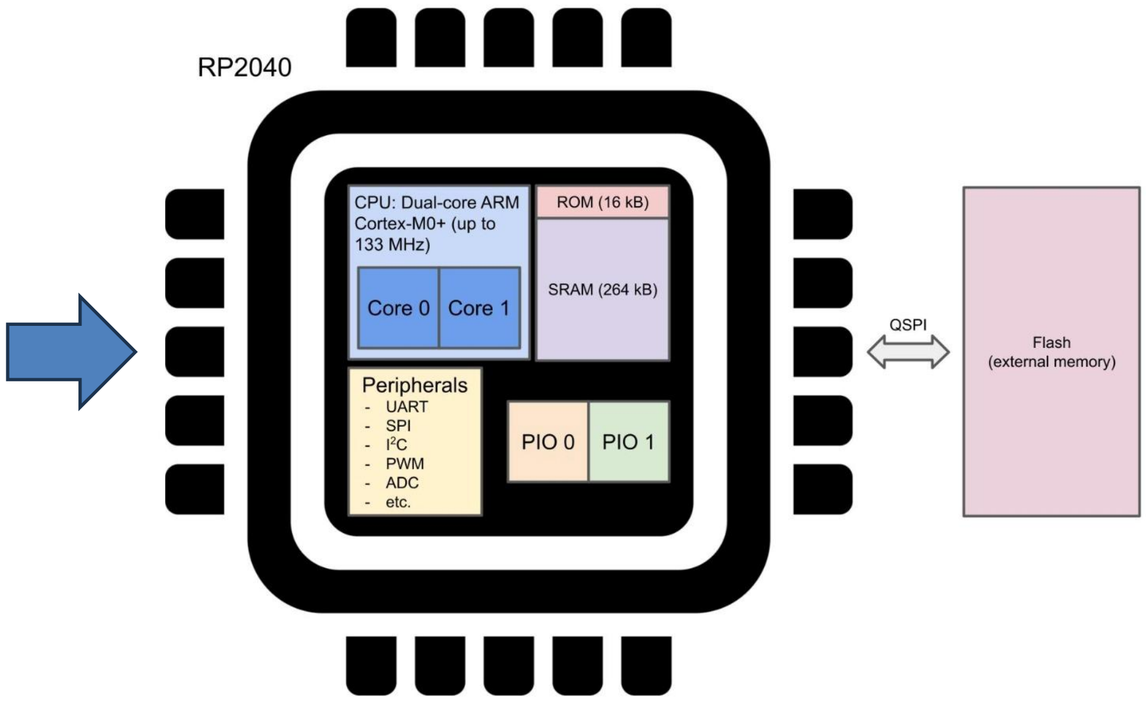
Arquitetura interna do microcontrolador rp2040.



Microcontrolador RP2040 - Revisão

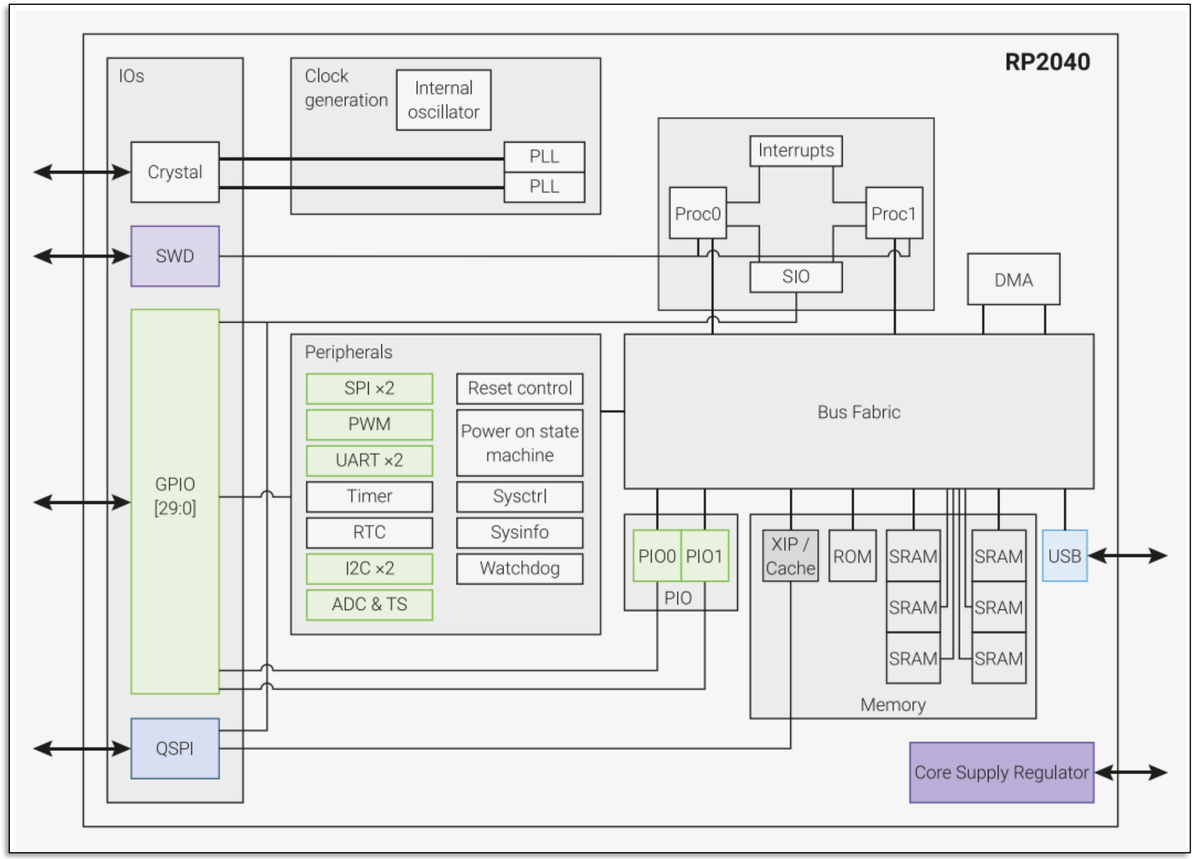
Arquitetura interna do microcontrolador rp2040.





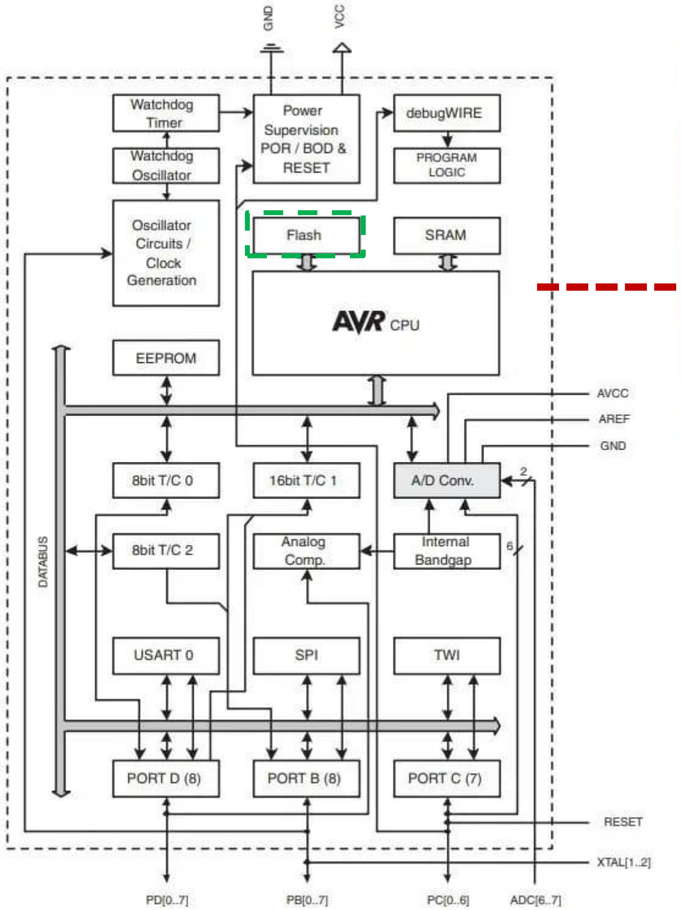
Microcontrolador RP2040 - Revisão

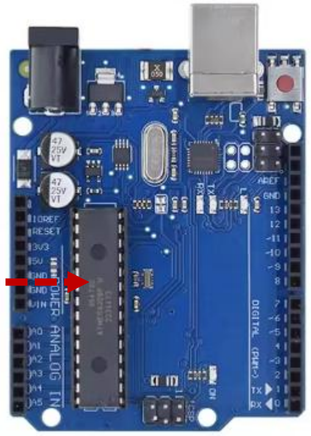
Arquitetura interna do microcontrolador rp2040.



VS

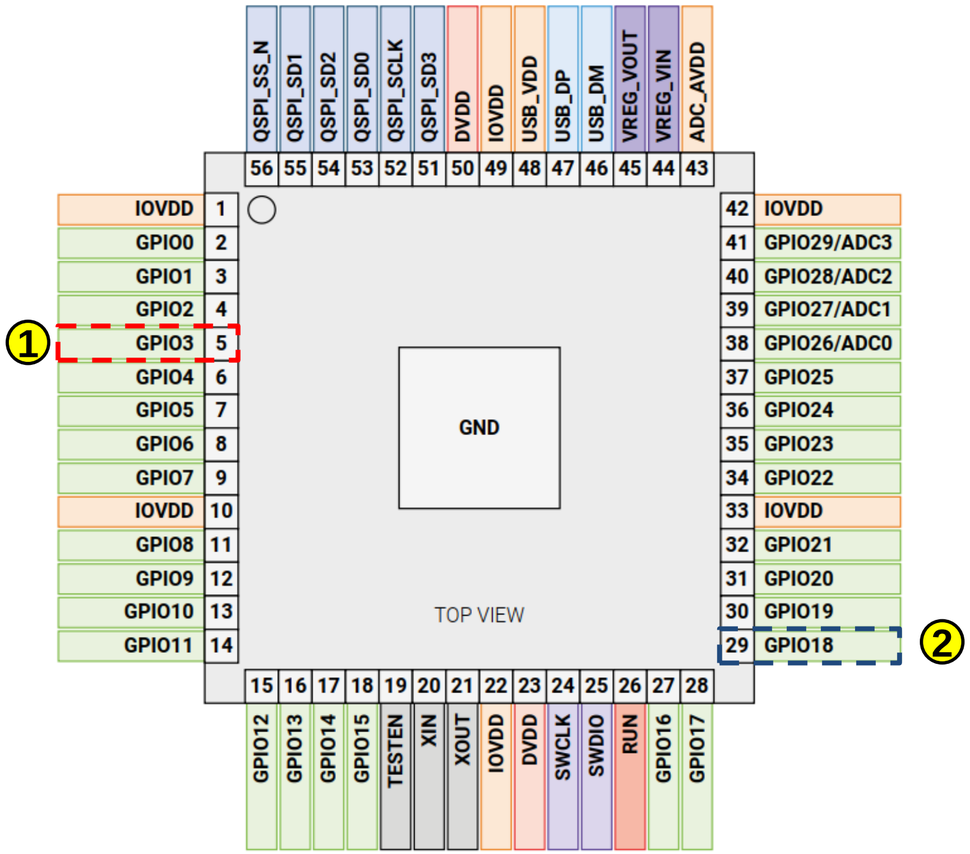
Arquitetura - atmega328p (Arduino Uno).





Microcontrolador RP2040 - Revisão

Pinagem do microcontrolador RP2040.



Funções da GPIO.

GPIO	F1	F2	F3	F4	F5	F6	F7	F8	F9
0	SPI0 RX	UART0 TX	I2C0 SDA	PWM0 A	SIO	PIO0	PIO1		USB OVCUR DET
1	SPI0 CSn	UART0 RX	I2C0 SCL	PWM0 B	SIO	PIO0	PIO1		USB VBUS DET
2	SPI0 SCK	UART0 CTS	I2C1 SDA	PWM1 A	SIO	PIO0	PIO1		USB VBUS EN
3	SPI0 TX	UART0 RTS	I2C1 SCL	PWM1 B	SIO	PIO0	PIO1		USB OVCUR DET
4	SPI0 RX	UART1 TX	I2C0 SDA	PWM2 A	SIO	PIO0	PIO1		USB VBUS DET
5	SPI0 CSn	UART1 RX	I2C0 SCL	PWM2 B	SIO	PIO0	PIO1		USB VBUS EN
13	SPI1 CSn	UART0 RX	I2C0 SCL	PWM6 B	SIO	PIO0	PIO1		USB VBUS DET
14	SPI1 SCK	UART0 CTS	I2C1 SDA	PWM7 A	SIO	PIO0	PIO1		USB VBUS EN
15	SPI1 TX	UART0 RTS	I2C1 SCL	PWM7 B	SIO	PIO0	PIO1		USB OVCUR DET
16	SPI0 RX	UART0 TX	I2C0 SDA	PWM0 A	SIO	PIO0	PIO1		USB VBUS DET
17	SPI0 CSn	UART0 RX	I2C0 SCL	PWM0 B	SIO	PIO0	PIO1		USB VBUS EN
18	SPI0 SCK	UART0 CTS	I2C1 SDA	PWM1 A	SIO	PIO0	PIO1		USB OVCUR DET
19	SPI0 TX	UART0 RTS	I2C1 SCL	PWM1 B	SIO	PIO0	PIO1		USB VBUS DET

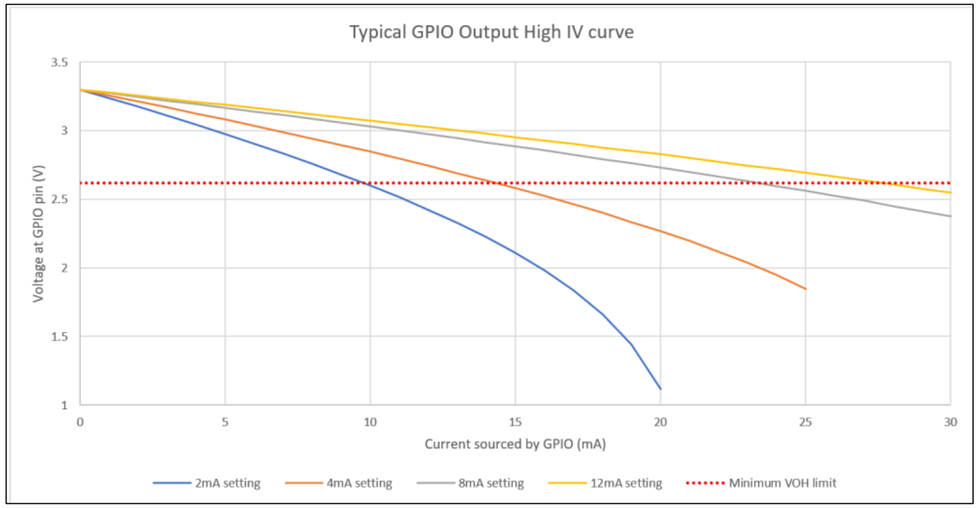
1

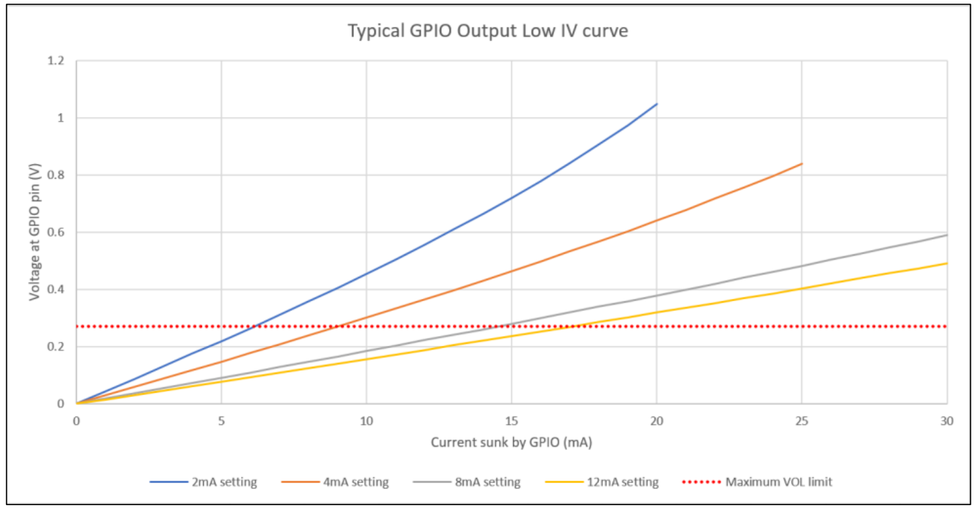
2

Microcontrolador RP2040 - Revisão

- Cuidados com os pinos GPIO do microcontrolador:

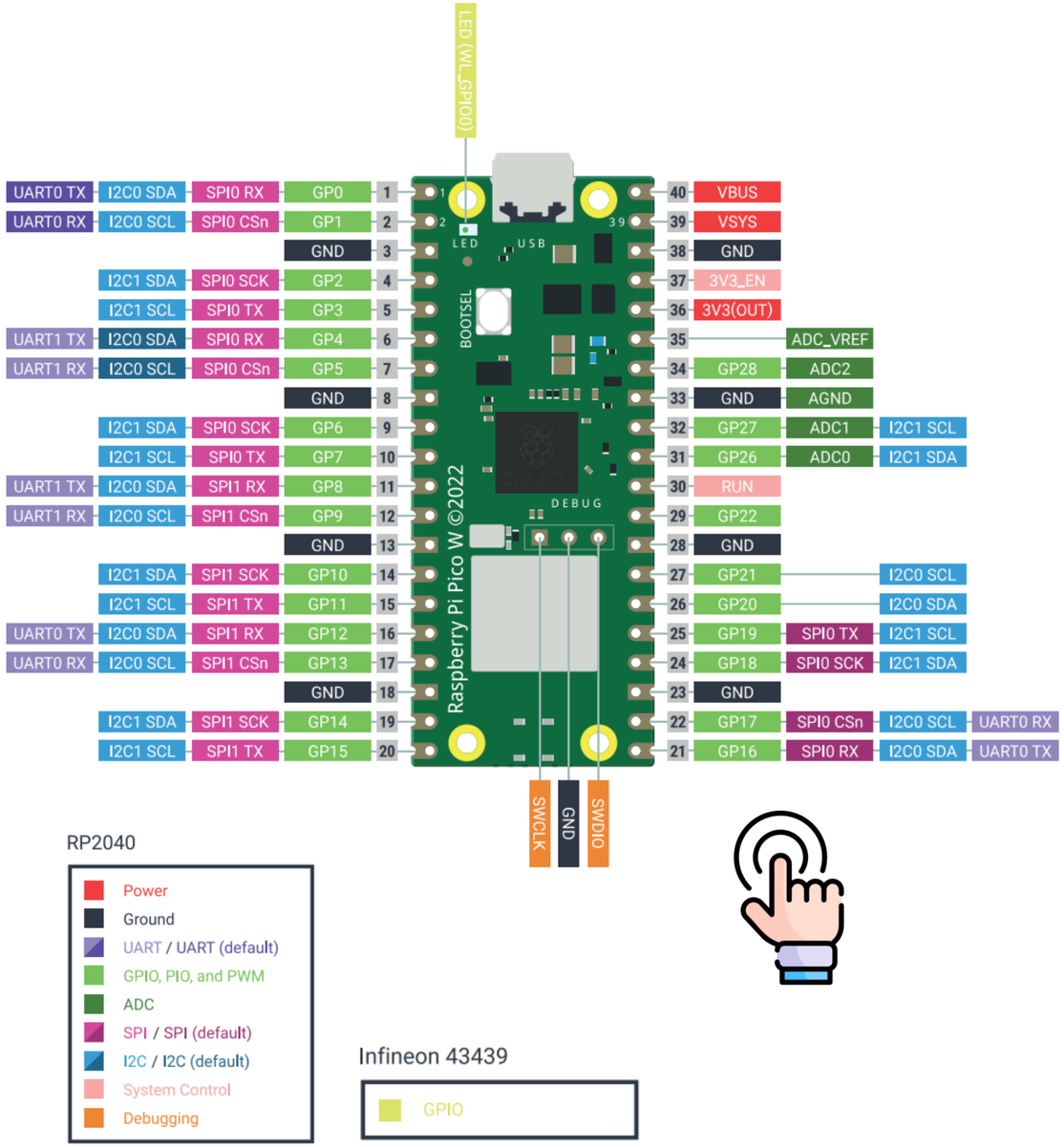
– **Limites de Tensão e Corrente**: Cada pino possui um limite máximo de tensão e corrente (*drive strength* - 2mA, 4mA, 8mA e 12mA) - IIOVSS_MAX (50mA). Avalie a utilização de circuitos de interface para cada tipo de periférico externo.





Raspberry Pi Pico W

- Expõe 26 E/S de uso geral (GPIO) multifuncionais de 3,3 V.



BitDogLab

BitDogLab

BitDogLAb (V 5.3.1).

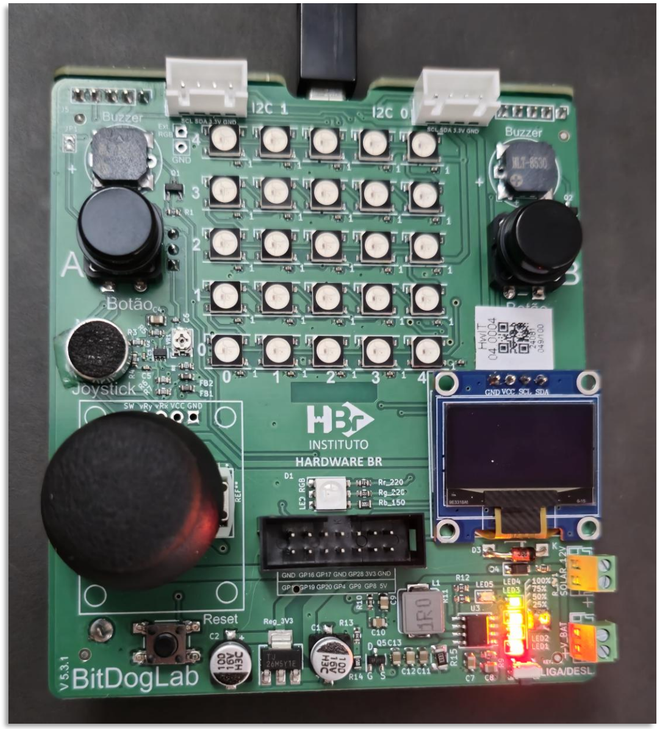
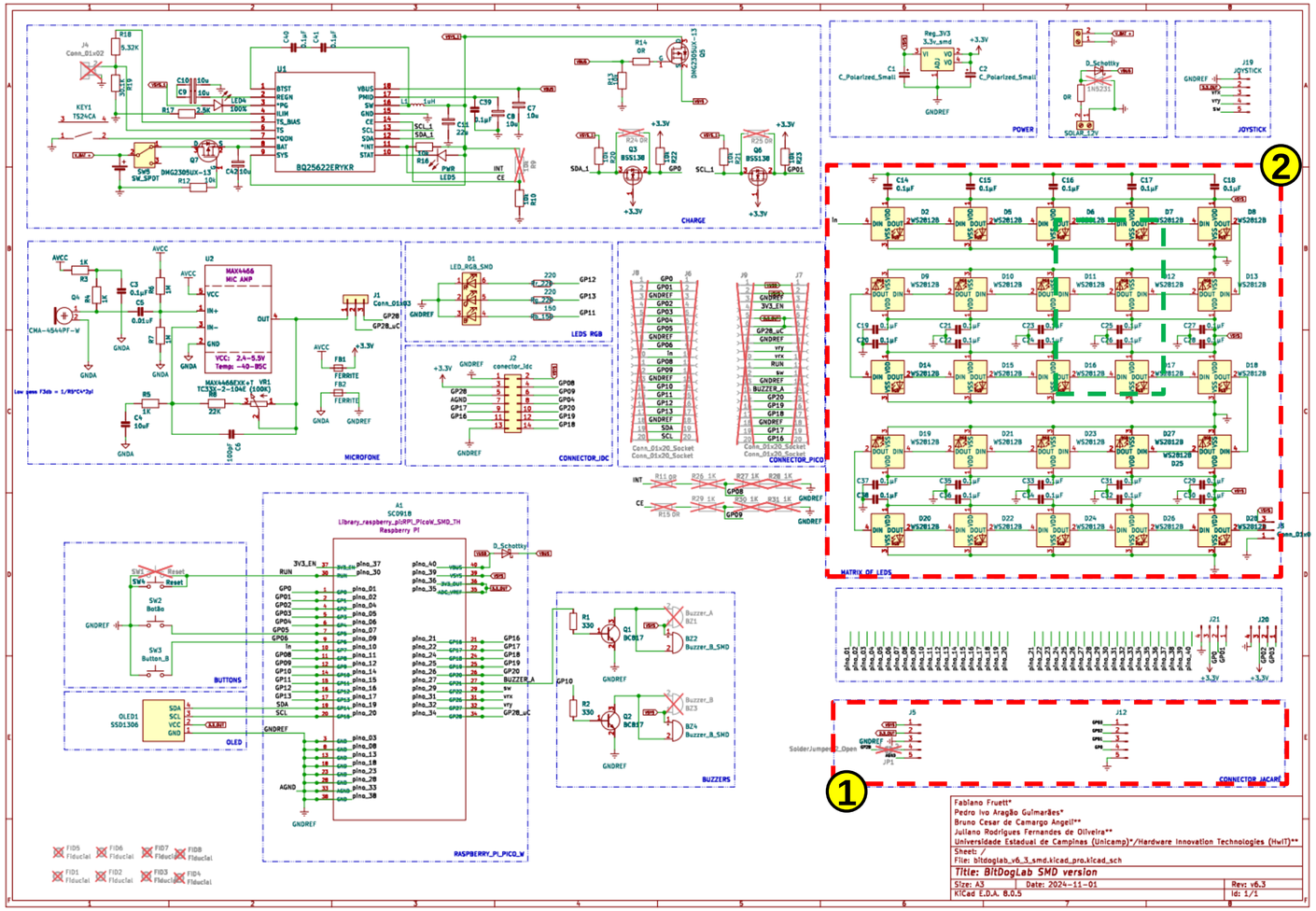


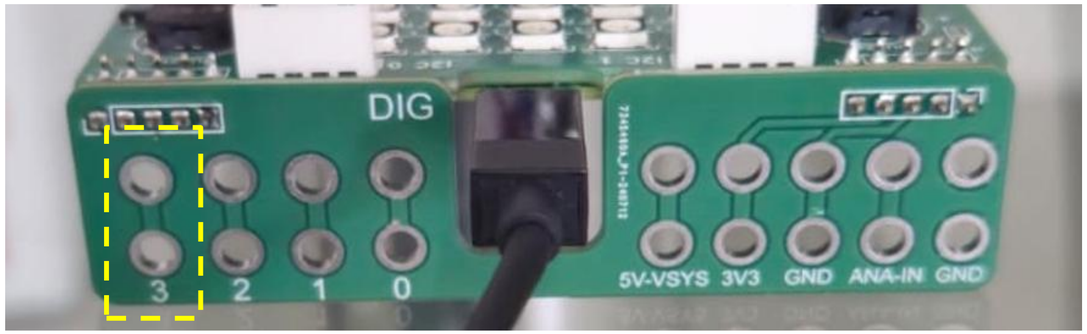
Diagrama esquemático da placa BitDogLAb (V 6.3).



BitDogLab

- Periféricos para estudo da GPIO:

1. Pinos e expansão para painel compatível com garras jacaré ou parafusos.

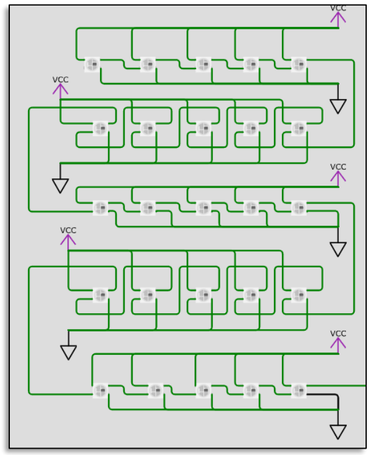


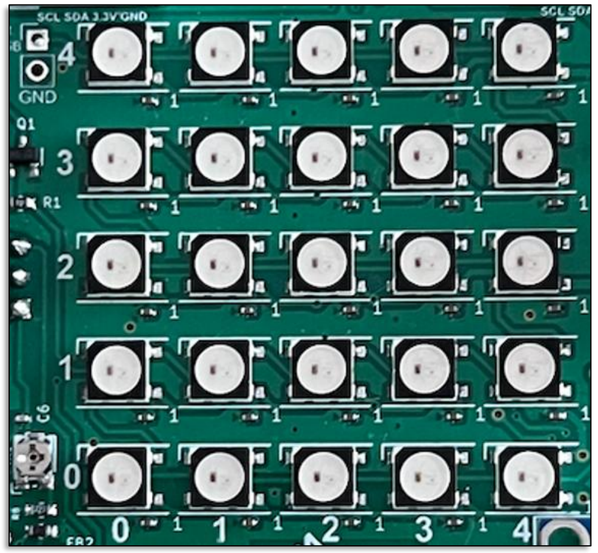
BitDogLab

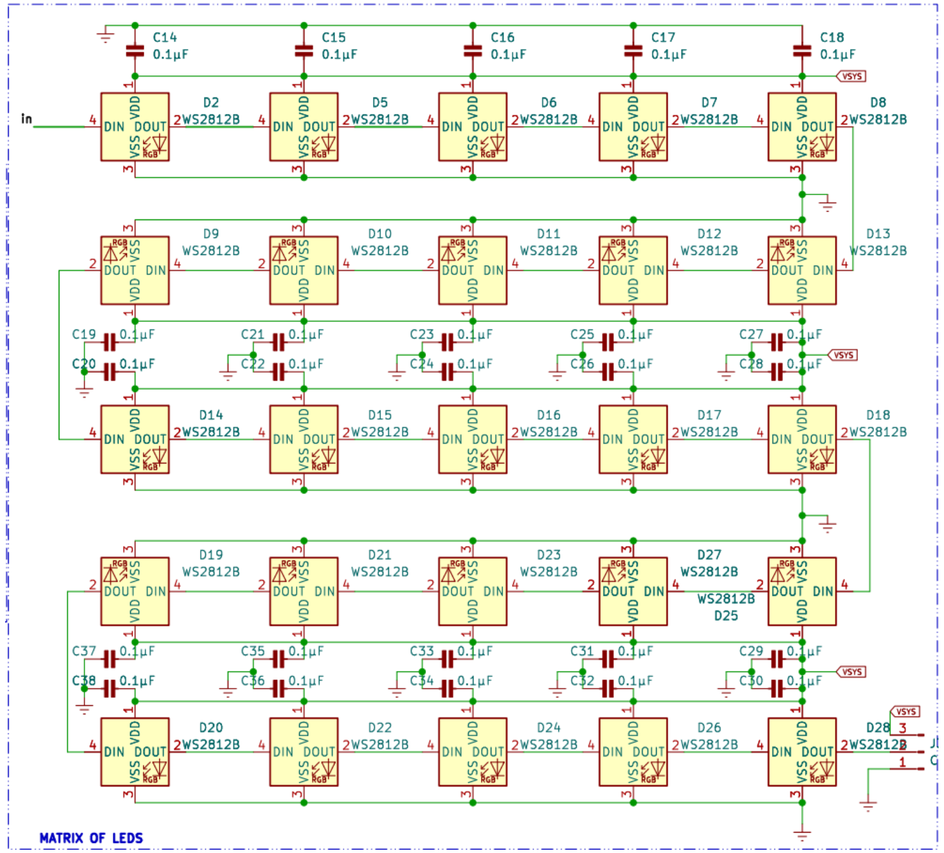
- Periféricos para estudo da GPIO:

2. Matriz de LEDs coloridos (LED-RGB 5x5 5050 WS2812)

- Conectado ao pino **GPIO 07**.



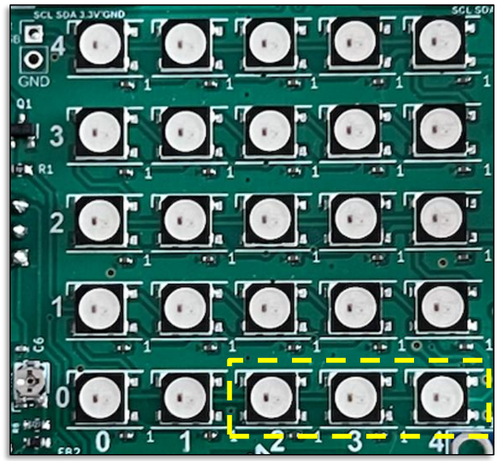


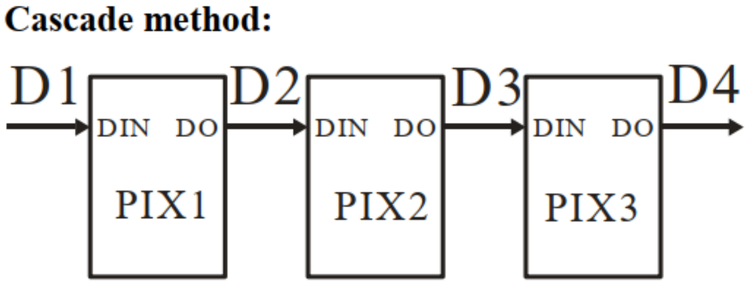


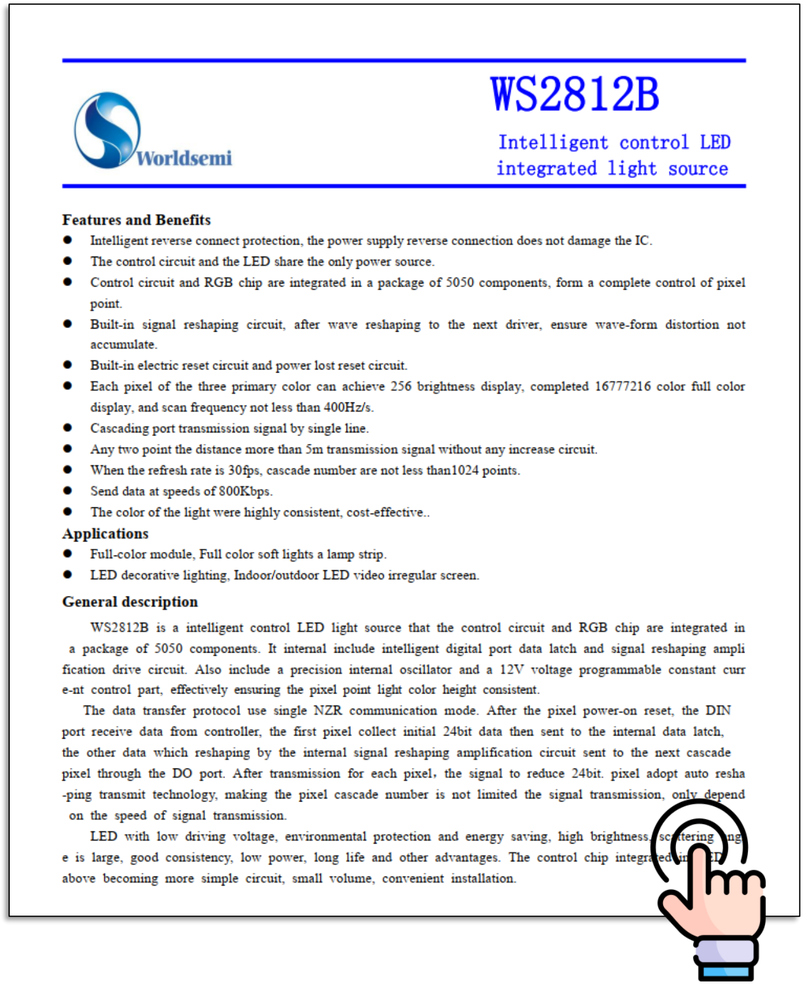
BitDogLab

- Periféricos para estudo da GPIO:

2. Matriz de LEDs coloridos (LED-RGB 5x5 5050 WS2812)







Worldsemi

WS2812B

Intelligent control LED integrated light source

Features and Benefits

- Intelligent reverse connect protection, the power supply reverse connection does not damage the IC.
- The control circuit and the LED share the only power source.
- Control circuit and RGB chip are integrated in a package of 5050 components, form a complete control of pixel point.
- Built-in signal reshaping circuit, after wave reshaping to the next driver, ensure wave-form distortion not accumulate.
- Built-in electric reset circuit and power lost reset circuit.
- Each pixel of the three primary color can achieve 256 brightness display, completed 16777216 color full color display, and scan frequency not less than 400Hz/s.
- Cascading port transmission signal by single line.
- Any two point the distance more than 5m transmission signal without any increase circuit.
- When the refresh rate is 30fps, cascade number are not less than1024 points.
- Send data at speeds of 800Kbps.
- The color of the light were highly consistent, cost-effective..

Applications

- Full-color module, Full color soft lights a lamp strip.
- LED decorative lighting, Indoor/outdoor LED video irregular screen.

General description

WS2812B is a intelligent control LED light source that the control circuit and RGB chip are integrated in a package of 5050 components. It internal include intelligent digital port data latch and signal reshaping ampli fication drive circuit. Also include a precision internal oscillator and a 12V voltage programmable constant curr e-nt control part, effectively ensuring the pixel point light color height consistent.

The data transfer protocol use single NZR communication mode. After the pixel power-on reset, the DIN port receive data from controller, the first pixel collect initial 24bit data then sent to the internal data latch, the other data which reshaping by the internal signal reshaping amplification circuit sent to the next cascade pixel through the DO port. After transmission for each pixel, the signal to reduce 24bit. pixel adopt auto resha -ping transmit technology, making the pixel cascade number is not limited the signal transmission, only depend on the speed of signal transmission.

LED with low driving voltage, environmental protection and energy saving, high brightness, scattering ang e is large, good consistency, low power, long life and other advantages. The control chip integrated in the LED above becoming more simple circuit, small volume, convenient installation.

BitDogLab

- Periféricos para estudo da GPIO:

2. Matriz de LEDs coloridos (LED-RGB 5x5 5050 WS2812)

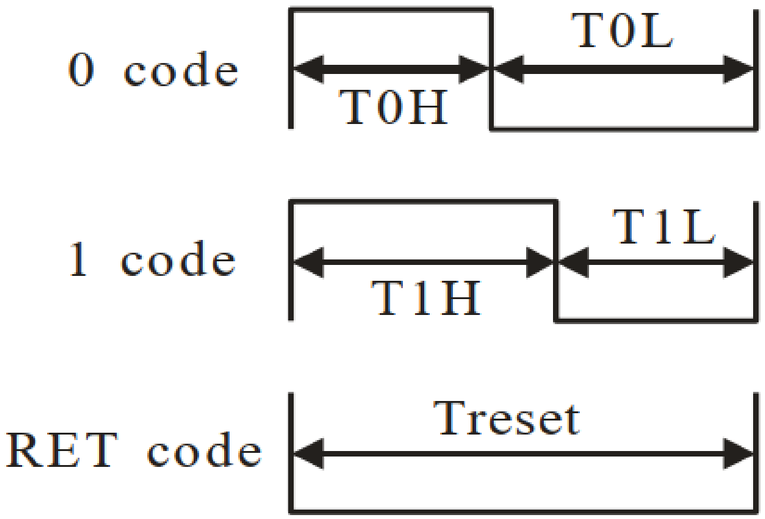
Data transfer time(TH+TL=1.25μs±600ns)

T0H	0 code ,high voltage time	0.4us	±150ns
T1H	1 code ,high voltage time	0.8us	±150ns
T0L	0 code , low voltage time	0.85us	±150ns
T1L	1 code ,low voltage time	0.45us	±150ns
RES	low voltage time	Above 50μs	

Composition of 24bit data:

G7	G6	G5	G4	G3	G2	G1	G0	R7	R6	R5	R4	R3	R2	R1	R0	B7	B6	B5	B4	B3	B2	B1	B0

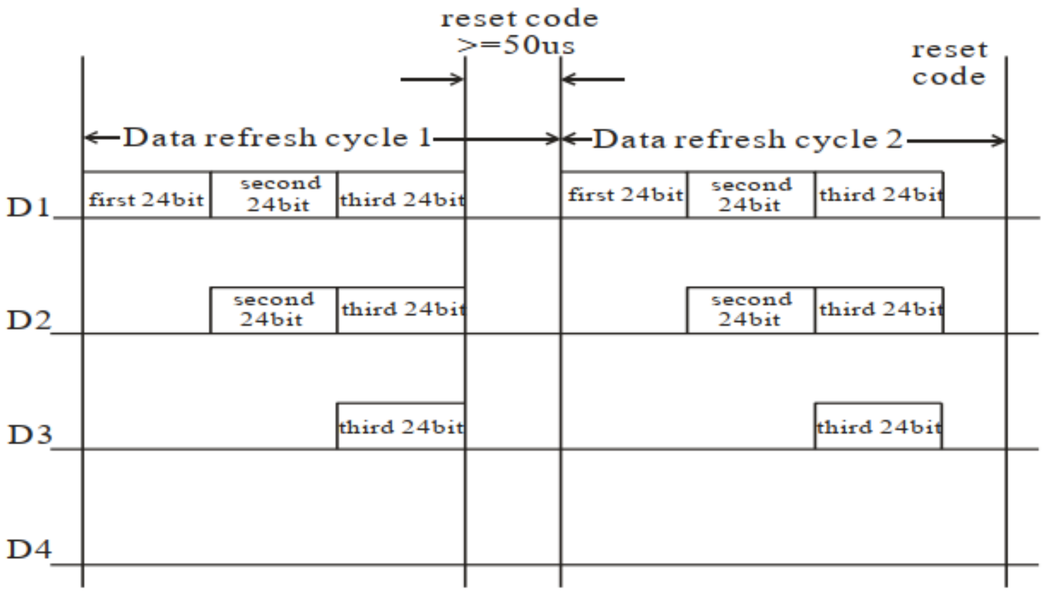
Note: Follow the order of GRB to sent data and the high bit sent at first.

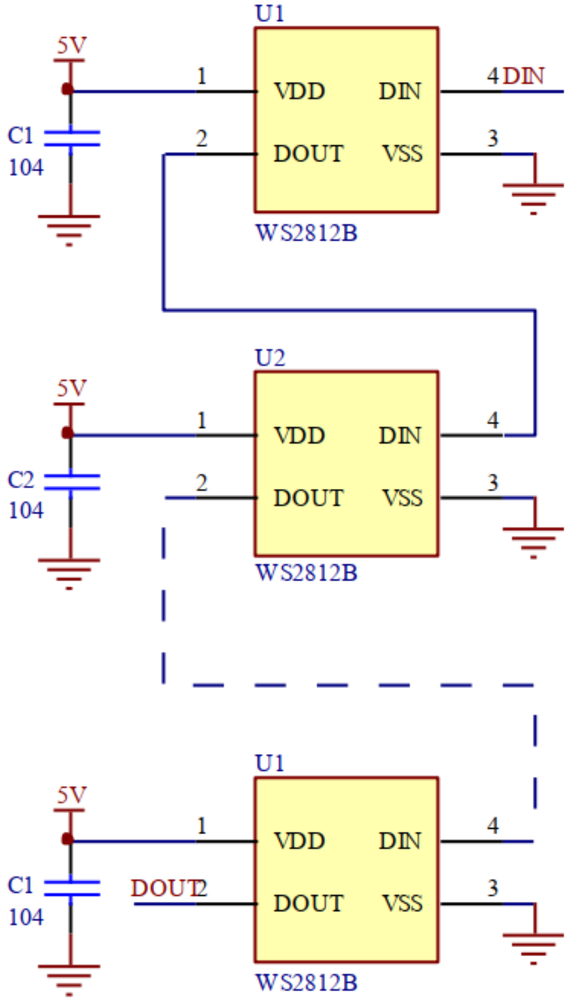


BitDogLab

- Periféricos para estudo da GPIO:

2. Matriz de LEDs coloridos (LED-RGB 5x5 5050 WS2812)





Pico SDK

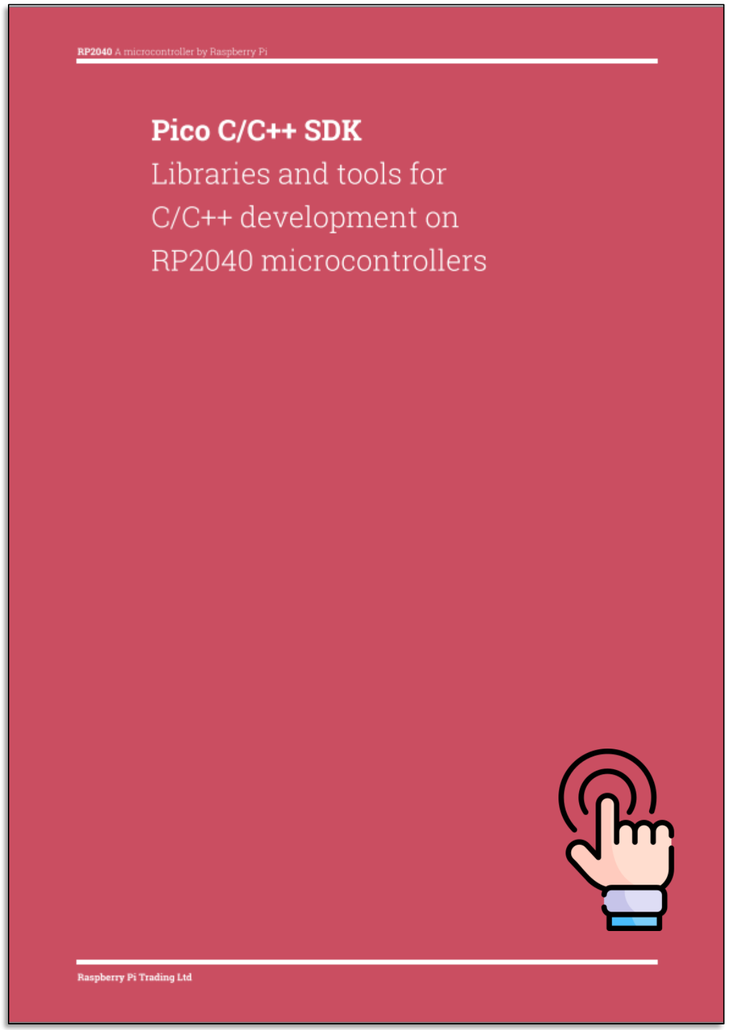
Pico SDK

raspberrypi/**pico-sdk**

- O **SDK** (*Software Development Kit* – Kit de desenvolvimento de software), fornece as bibliotecas e sistema de compilação necessários para programar dispositivos baseados em RP2040, em linguagem C, C++ ou Assembly Arm.

- Fornece um ambiente de programação familiar para os desenvolvedores C.

- Possui bibliotecas de nível alto, mas também conta recursos para acessar o *hardware* bruto.



Pico SDK

- Sleep (4.2.10.) / Function list (4.2.10.1) – rotinas de atrasos.

1. void sleep_ms (uint32_t ms) - Espera por um certo número de milissegundos antes de retornar.

```
while (true)
  {
    gpio_put(led_pin_red, true); //ligar led
    sleep_ms(10); //10 milissegundos.
    gpio_put(led_pin_red, false); //desligar led
    sleep_ms(10);
  }
```

Pico SDK

- Sleep (4.2.10.) / Function list (4.2.10.1) – rotinas de atrasos.

1. void sleep_us (uint64_t us) - Espera por um certo número de microssegundos antes de retornar.

```
while (true)
  {
    gpio_put(led_pin_red, true); //ligar led
    sleep_us(10); //10 microssegundos.
    gpio_put(led_pin_red, false); //desligar led
    sleep_us(10);
  }
```

Pico SDK

- Sleep (4.2.10.) / Function list (4.2.10.1) – rotinas de atrasos.

1. void sleep_until (absolute_time_t target) - Espera até depois do carimbo de tempo (*timestamp*) indicado para regressar.

```
gpio_put(led_pin_red,true); //ligar led

//absolute_time_t target_time = get_absolute_time() + make_timeout_time_ms(1000);
absolute_time_t target_time = make_timeout_time_ms(1000); // carimbo de tempo
sleep_until(target_time); //dormir por 1 segundo

gpio_put(led_pin_red,false); //desligar led

```

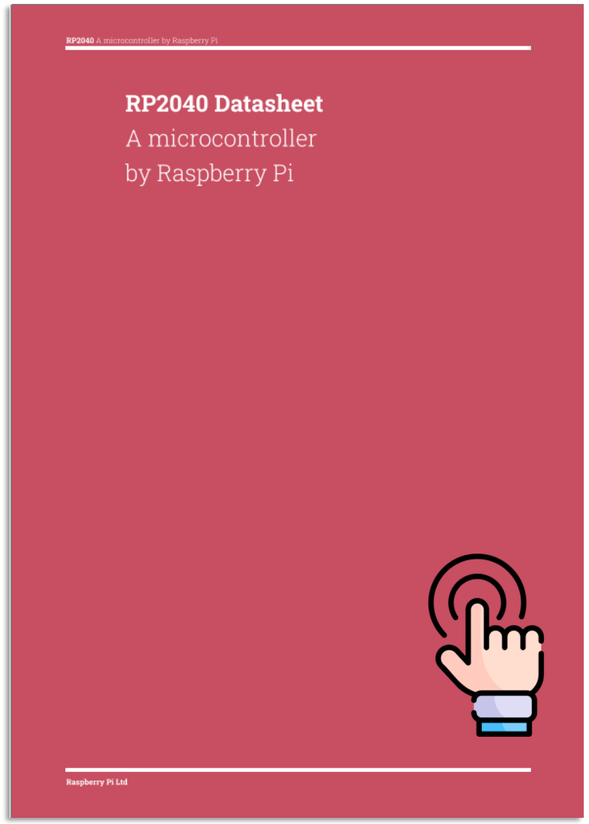
Introdução ao PIO

Introdução ao PIO

- O **PIO** (*Programmable I/O*) é um bloco de hardware desenvolvido para o microcontrolador RP2040.

- É capaz de gerar **formas de onda personalizadas**, como ondas quadradas, ondas senoidais e sinais PWM.

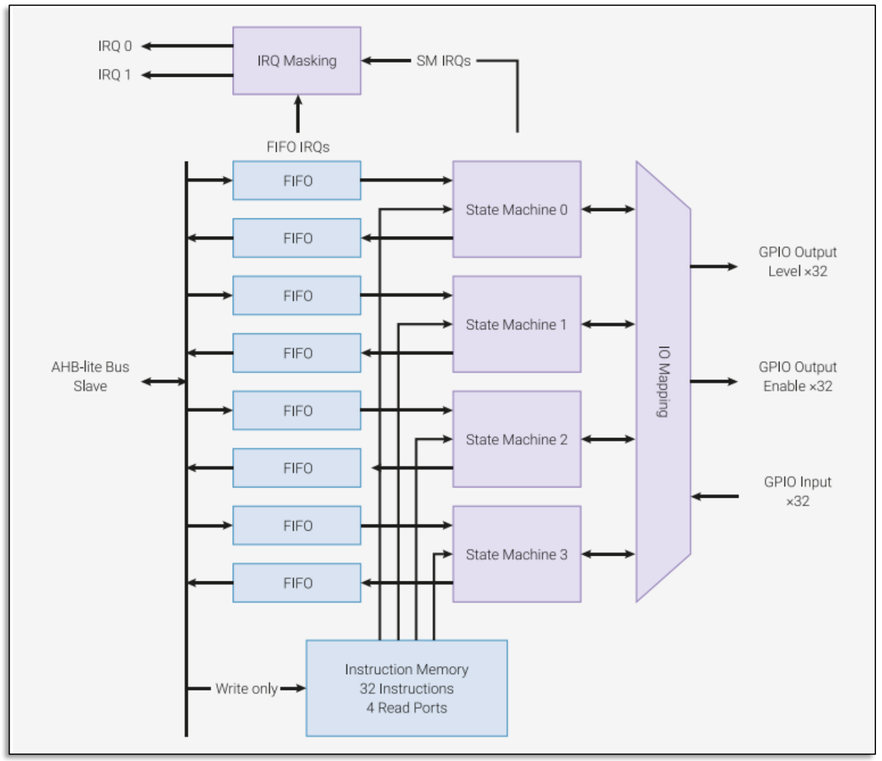
- Ele permite a criação de **novos tipos de interface de comunicação** – viabiliza comunicar com dispositivos externos de maneira que não foram previstas pelos projetistas do microcontrolador.



Introdução ao PIO

- O PIO pode ser usado para controlar LEDs RGB endereçáveis individualmente, como o WS2812.
- É capaz de capturar dados de alta velocidade de sensores ou outros dispositivos.

Diagrama de um bloco PIO:

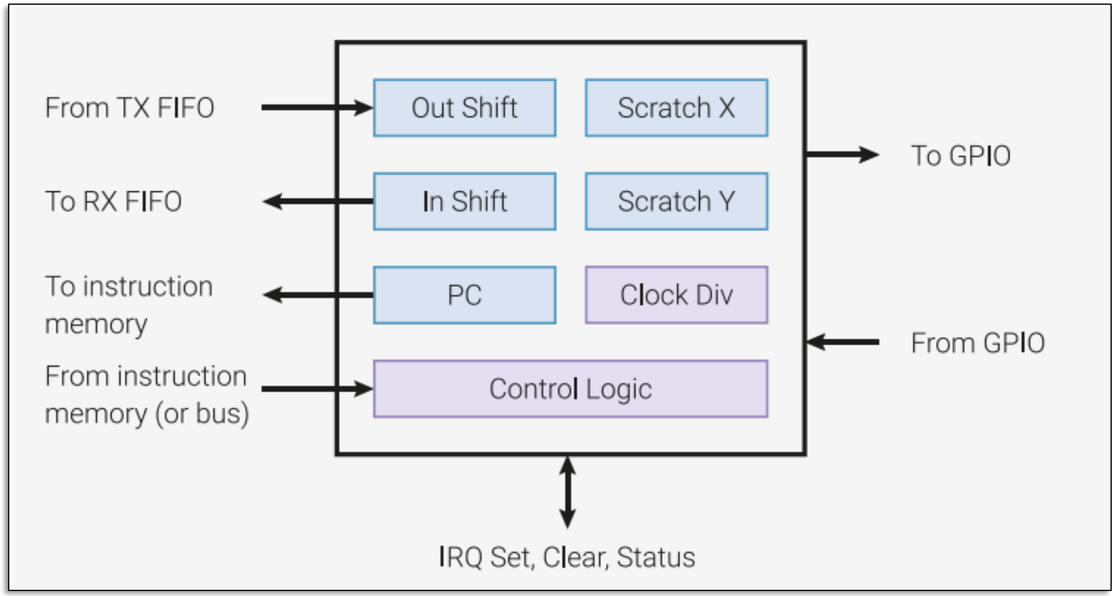


Introdução ao PIO

- Suporta uma grande variedade de interfaces de comunicação:

- 8080 and 6800 parallel bus
- I2C
- 3-pin I2S
- SDIO
- SPI, DSPI, QSPI
- UART
- DPI or VGA (via resistor DAC)

Detalhamento da Máquina de Estados.



Introdução ao PIO

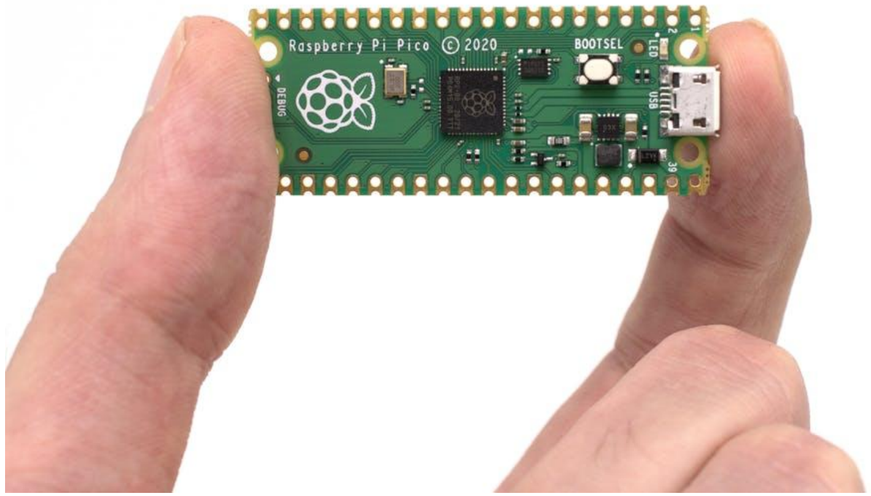
- Todas as instruções do PIO executam em apenas um ciclo.

Bit:	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JMP	0	0	0	Delay/side-set					Condition			Address				
WAIT	0	0	1	Delay/side-set					Pol	Source		Index				
IN	0	1	0	Delay/side-set					Source			Bit count				
OUT	0	1	1	Delay/side-set					Destination			Bit count				
PUSH	1	0	0	Delay/side-set					0	IfF	Blk	0	0	0	0	0
PULL	1	0	0	Delay/side-set					1	IfE	Blk	0	0	0	0	0
MOV	1	0	1	Delay/side-set					Destination			Op		Source		
IRQ	1	1	0	Delay/side-set					0	Clr	Wait	Index				
SET	1	1	1	Delay/side-set					Destination			Data				

Exemplos de aplicação

Vamos para o microcontrolador!

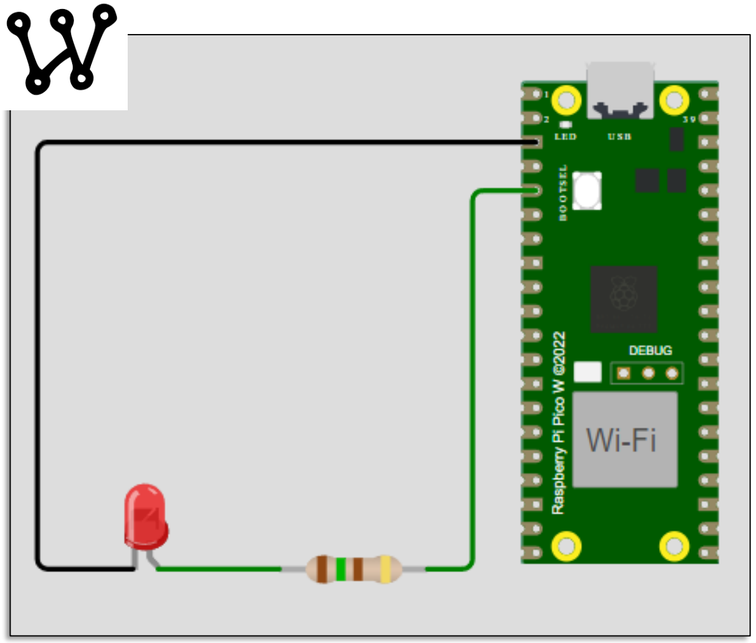
Prática 1



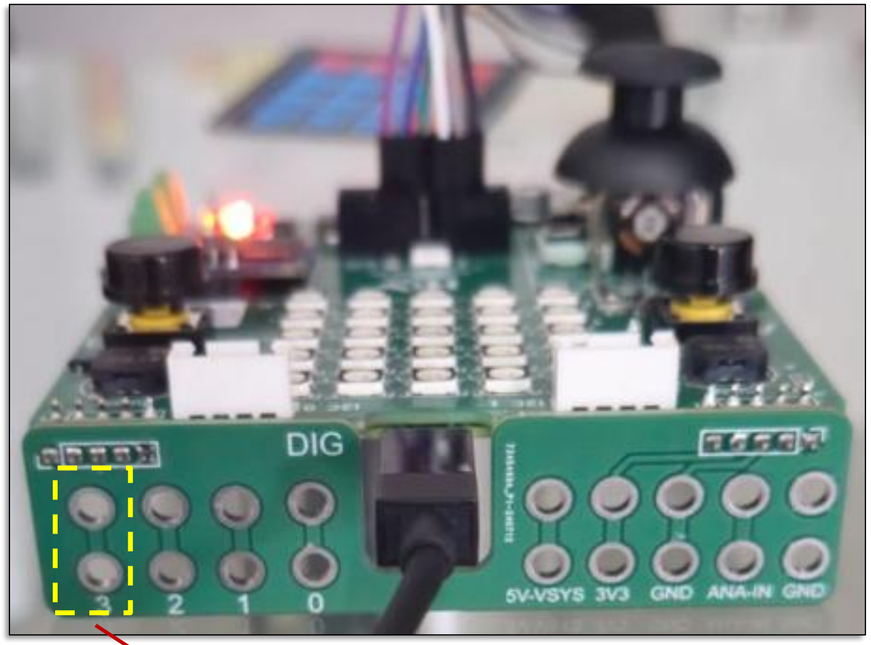
Prática 1 – Manipulação de GPIO

Acionamento de saída digital (GPIO 03) com diferentes atrasos (*delays*).

Simulação:



Hardware BitDogLab:



GPIO 03

Osciloscopio

Prática 1 – Manipulação de GPIO

Lembrete: Organização básica do código fonte.

```
1    #include <stdio.h>
2    #include "pico/stdlib.h" //biblioteca SDK padrão do PICO - funcionalidades para programação básica.
3
4    // Definição de macros ou variáveis/constantes globais (1)
5
6    void main()
7    {
8         // inicializações de hardware e software (2)
9
10        while(true) // loop infinito
11        {
12             /* tarefas a serem executadas pelo microcontrolador durante o funcionamento */ (3)
13        }
14   }
```

Prática 1 – Manipulação de GPIO

Código fonte:

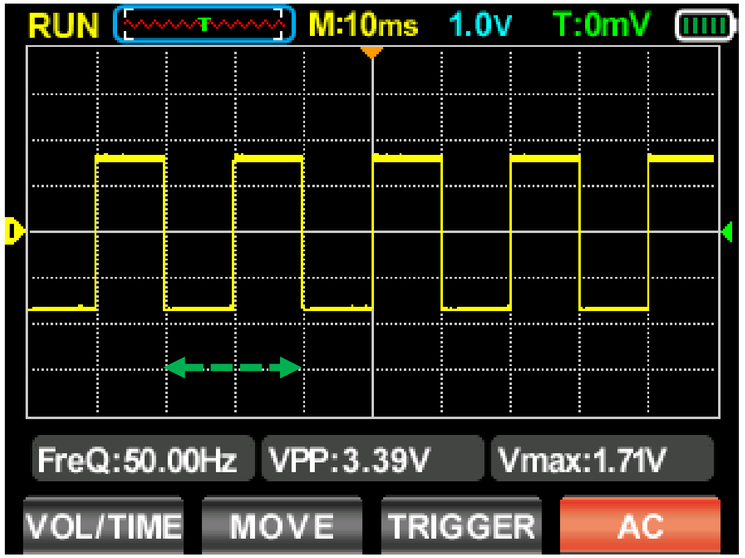
```c
//Rotina para acionar portas digitais do raspberry pi pico w - piscar LED
#include <stdio.h>
#include "pico/stdlib.h" //biblioteca SDK padrão do PICO 4

const uint led_pin_red = 03;

int main()
{

  gpio_init(led_pin_red); //inicializar a porta digital do microcontrolador
  gpio_set_dir(led_pin_red, GPIO_OUT); //Configurar a porta como saída.

  stdio_init_all(); // Inicializa todos os códigos stdio padrão.

  while (true)
  {

    gpio_put(led_pin_red, true); //ligar led
    sleep_ms(10); //delay de 10ms
    gpio_put(led_pin_red, false); //desligar led
    sleep_ms(10); //delay de 10ms

  }
}
```

1

2

3

Tensão na GPIO03.



$$T = 20\text{ms} = 2 \times 10^{-2}\ s$$

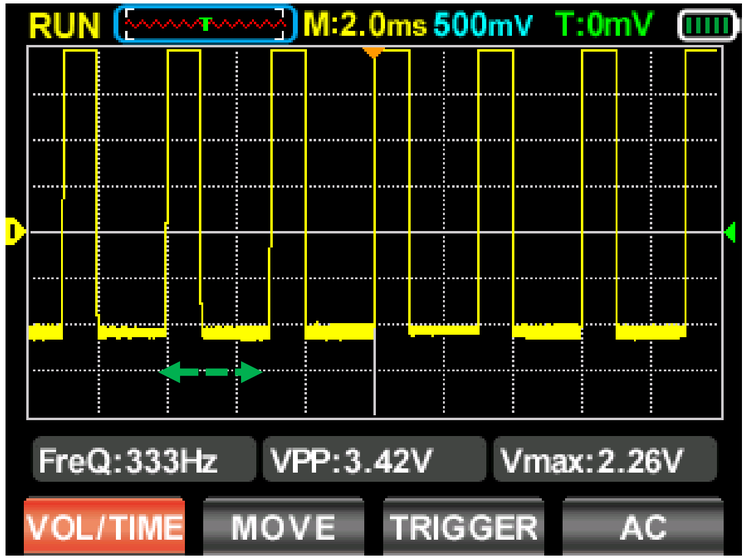
$$f = \frac{1}{T} = \frac{1}{2 \times 10^{-2}} = 50\text{Hz}$$

Prática 1 – Manipulação de GPIO

Código fonte:

```c
//Rotina para acionar portas digitais do raspberry pi pico w - piscar LED
#include <stdio.h>
#include "pico/stdlib.h" //biblioteca SDK padrão do PICO 4

const uint led_pin_red = 03;

int main()
{

  gpio_init(led_pin_red); //inicializar a porta digital do microcontrolador
  gpio_set_dir(led_pin_red, GPIO_OUT); //Configurar a porta como saída.

  stdio_init_all(); // Inicializa todos os códigos stdio padrão.

  while (true)
  {

    gpio_put(led_pin_red, true); //ligar led
    sleep_ms(1); //delay de 1ms
    gpio_put(led_pin_red, false); //desligar led
    sleep_ms(2); //delay de 2ms

  }
}
```

Tensão na GPIO03.



$$T = 3 \times 10^{-3}\ s$$

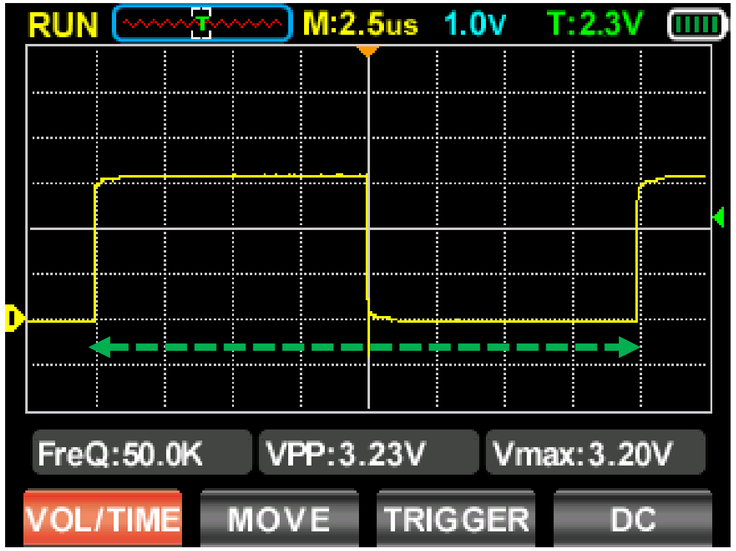
$$f = \frac{1}{T} = \frac{1}{3 \times 10^{-3}} \cong 333\text{Hz}$$

Prática 1 – Manipulação de GPIO

Código fonte:

```c
//Rotina para acionar portas digitais do raspberry pi pico w - piscar LED
#include <stdio.h>
#include "pico/stdlib.h" //biblioteca SDK padrão do PICO 4

const uint led_pin_red = 03;

int main()
{

  gpio_init(led_pin_red); //inicializar a porta digital do microcontrolador
  gpio_set_dir(led_pin_red, GPIO_OUT); //Configurar a porta como saída.

  stdio_init_all(); // Inicializa todos os códigos stdio padrão.

  while (true)
  {

    gpio_put(led_pin_red, true); //ligar led
    sleep_us(1); //delay de 1us
    gpio_put(led_pin_red, false); //desligar led
    sleep_us(2); //delay de 1us

  }
}
```

Tensão na GPIO03.

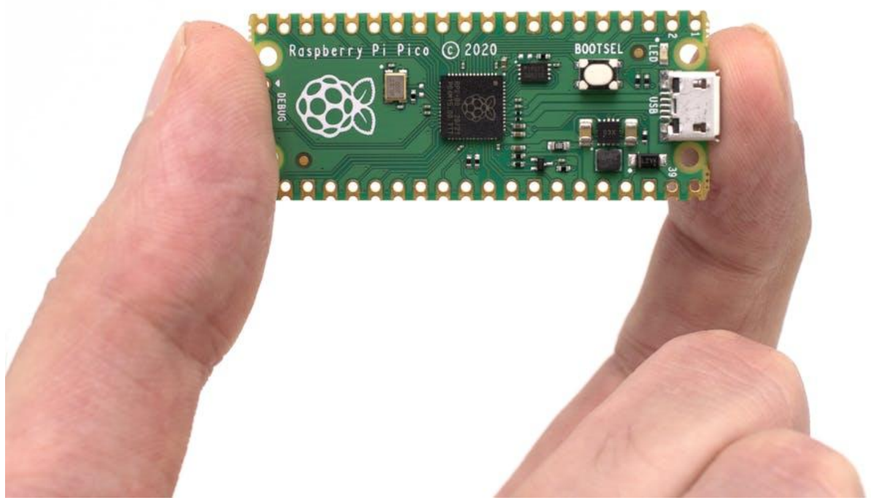


$$T = 20\mu s = 2 \times 10^{-5}\ s$$

$$f = \frac{1}{T} = \frac{1}{2 \times 10^{-5}} \cong 50\text{kHz}$$

Vamos para o microcontrolador!

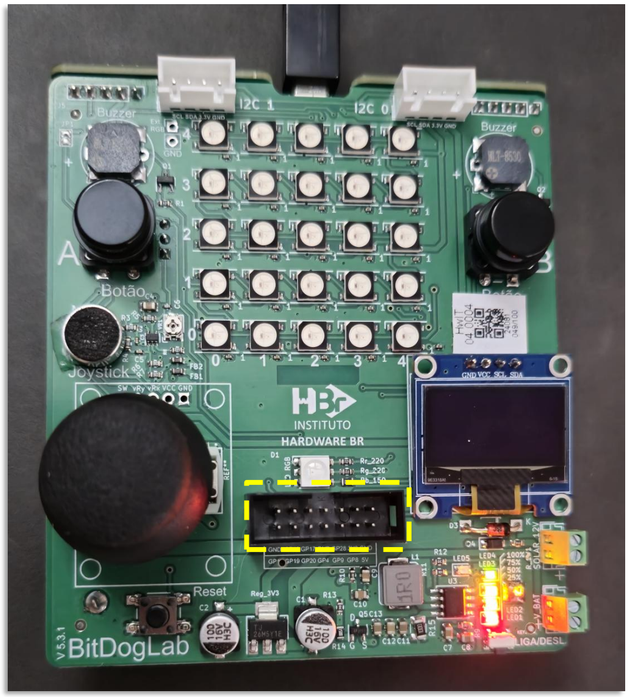
Prática 2



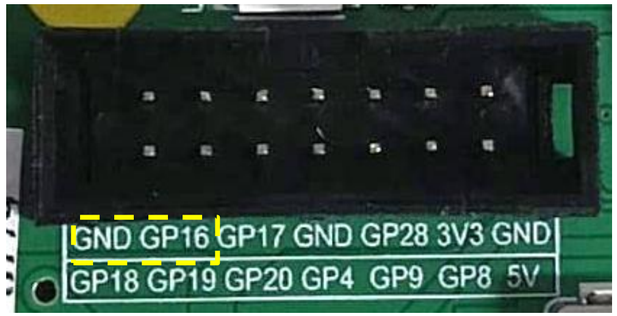
Prática 2 – PIO / GPIO em Alta Frequência

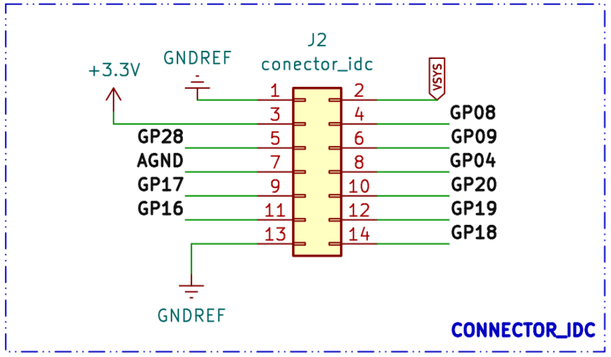
Geração de onda quadrada com período de 200ns – utiliza o periférico PIO 0.

Hardware BitDogLab:

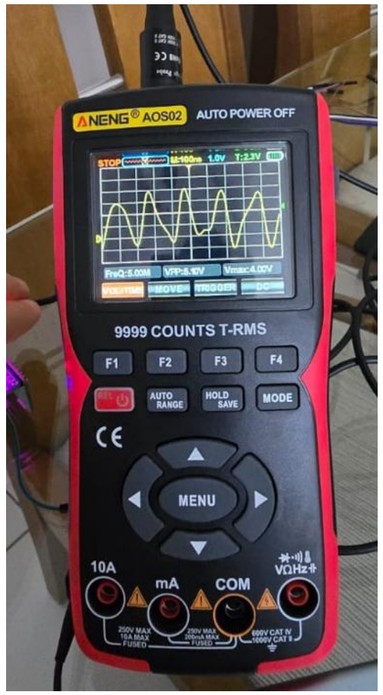


Conector de expansão de GPIOs:





Osciloscopio



Prática 2 – PIO / GPIO em Alta Frequência

Código fonte:

```c
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/pio.h"
#include "pio_wave.pio.h"
#include "hardware/clocks.h"

#define PIN 16 //Definição do pino de saída

int main(){ //função principal

    PIO pio = pio0; //especifica o periférico pio0
    bool ok;

    //coloca a frequência de clock para 128 MHz, facilitando a divisão pelo clock
    ok = set_sys_clock_khz(128000, false);

    stdio_init_all();

    printf("iniciando a transmissão PIO");
    if (ok) printf("clock set to %ld\n", clock_get_hz(clk_sys));

    uint offset = pio_add_program(pio, &pio_wave_program); //configura a pio
    uint sm = pio_claim_unused_sm(pio, true);
    PIOMaxSqaureWave_program_init(pio, sm, offset, PIN, 12.80f);

    while(1){

        //loop infinito

    }
}
```

$$f_{PIO} = \frac{128 \times 10^6}{12{,}8} = 10MHz$$

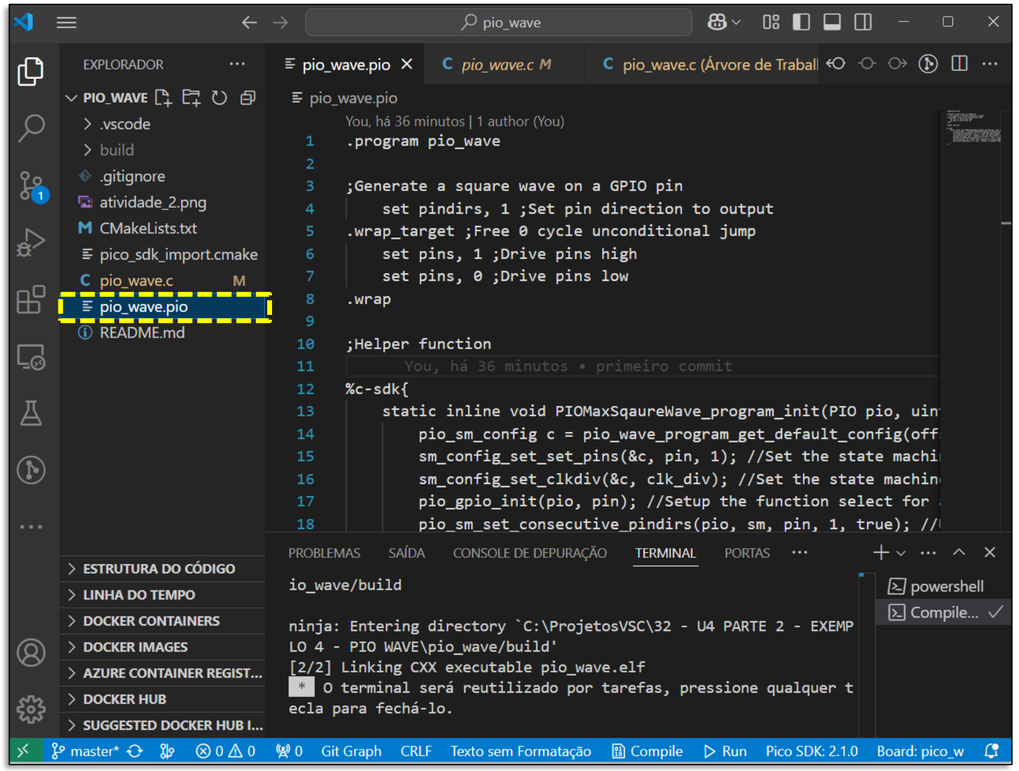
$$T_{PIO} = \frac{1}{f_{PIO}} = 100ns$$

Prática 2 – PIO / GPIO em Alta Frequência

Configuração da PIO – pio_wave.pio:

```
.program pio_wave

;Generate a square wave on a GPIO pin
    set pindirs, 1 ;Set pin direction to output
.wrap_target ;Free 0 cycle unconditional jump
    set pins, 1 ;Drive pins high
    set pins, 0 ;Drive pins low
.wrap

;Helper function

%c-sdk{
    static inline void PIOMaxSqaureWave_program_init(PIO pio, uint sm, uint offset, uint pin, float clk_div){
        pio_sm_config c = pio_wave_program_get_default_config(offset); //Get default configurations for the PIO state machine
        sm_config_set_set_pins(&c, pin, 1); //Set the state machine configurations on the given pin
        sm_config_set_clkdiv(&c, clk_div); //Set the state machine clock divider
        pio_gpio_init(pio, pin); //Setup the function select for a GPIO to use output from the given PIO instance
        pio_sm_set_consecutive_pindirs(pio, sm, pin, 1, true); //Use a state machine to set the same pin direction for multiple
consecutive pins for the PIO instance
        pio_sm_init(pio, sm, offset, &c); //Resets the state machine to a consistent state, and configures it
        pio_sm_set_enabled(pio, sm, true); //Enable or disable a PIO state machine
    }
%}
```

Instruções em Assembly:



Configurações da PIO

Prática 2 – PIO / GPIO em Alta Frequência

CMakeLists.txt

EXPLORADOR

PIO_WAVE
- .vscode
- build
- .gitignore
- atividade_2.png
- CMakeLists.txt
- pico_sdk_import.cmake
- pio_wave.c M
- pio_wave.pio
- README.md

pio_wave.pio | CMakeLists.txt | pio_wave.c (Árvore de Trabalho) M

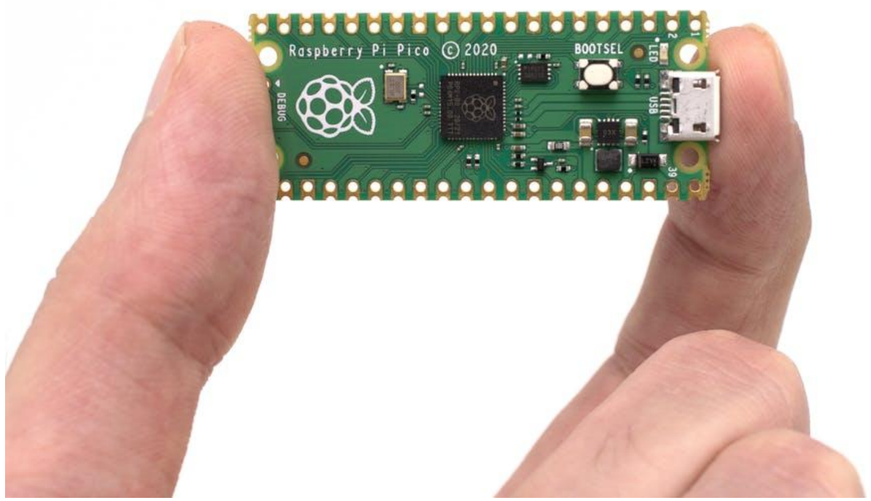
CMakeLists.txt

```
40  pico_set_program_name(pio_wave "pio_wave")
41  pico_set_program_version(pio_wave "0.1")
42
43  # Modify the below lines to enable/disable output over UART/USB
44  pico_enable_stdio_uart(pio_wave 0)
45  pico_enable_stdio_usb(pio_wave 1)
46
47  pico_generate_pio_header(pio_wave ${CMAKE_CURRENT_LIST_DIR}/pio_wave.pio)
48
49  target_sources(pio_wave PRIVATE pio_wave.c)
50
51  # Add the standard library to the build
52  target_link_libraries(pio_wave PRIVATE
53          pico_stdlib
54          hardware_pio
55          hardware_clocks)        You, há 39 minutos • primeiro commit
56
```

Tensão em GPIO 16:

Vamos para o microcontrolador!

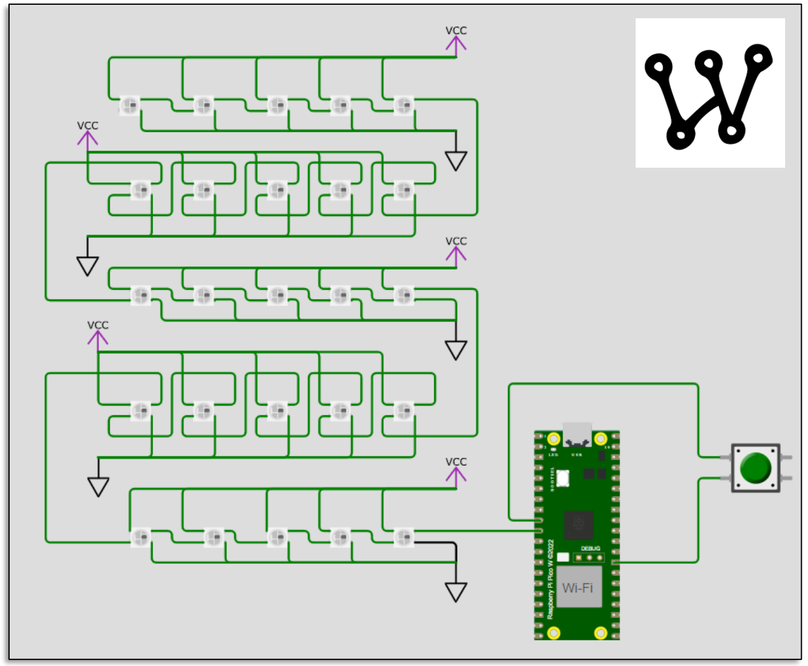
Prática 3



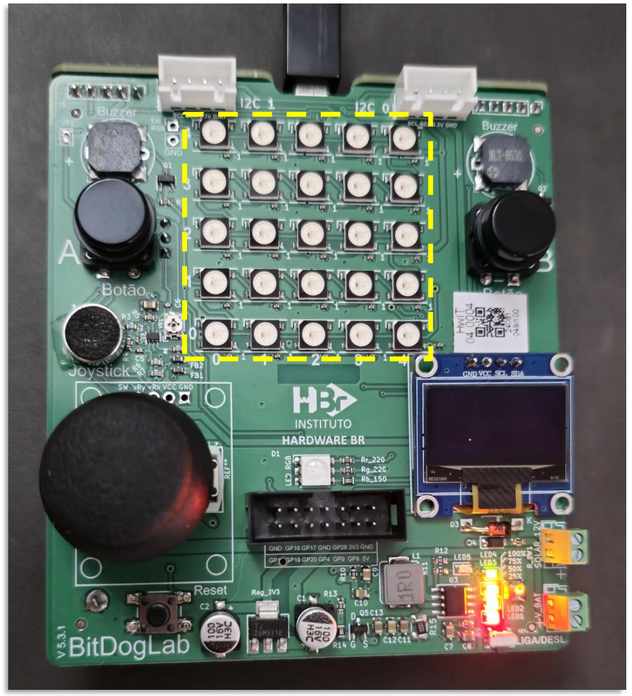
Prática 3 – Matrix de Leds (WS2812B)

Leitura de botão no GPIO 6 e acionamento da Matriz de Leds pela GPIO 07.

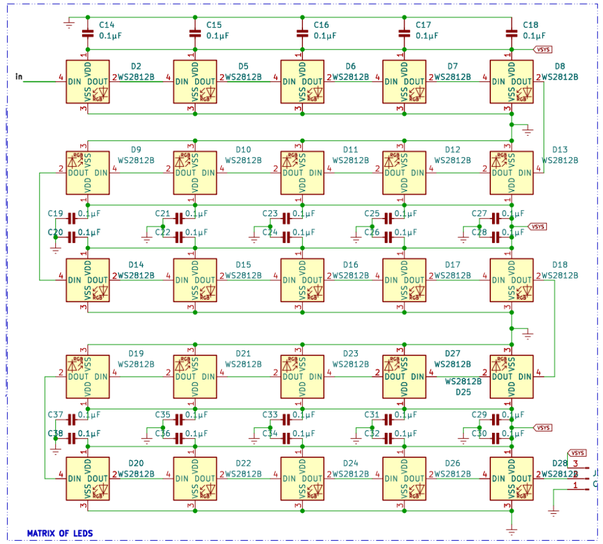
Simulação:



Hardware BitDogLab:

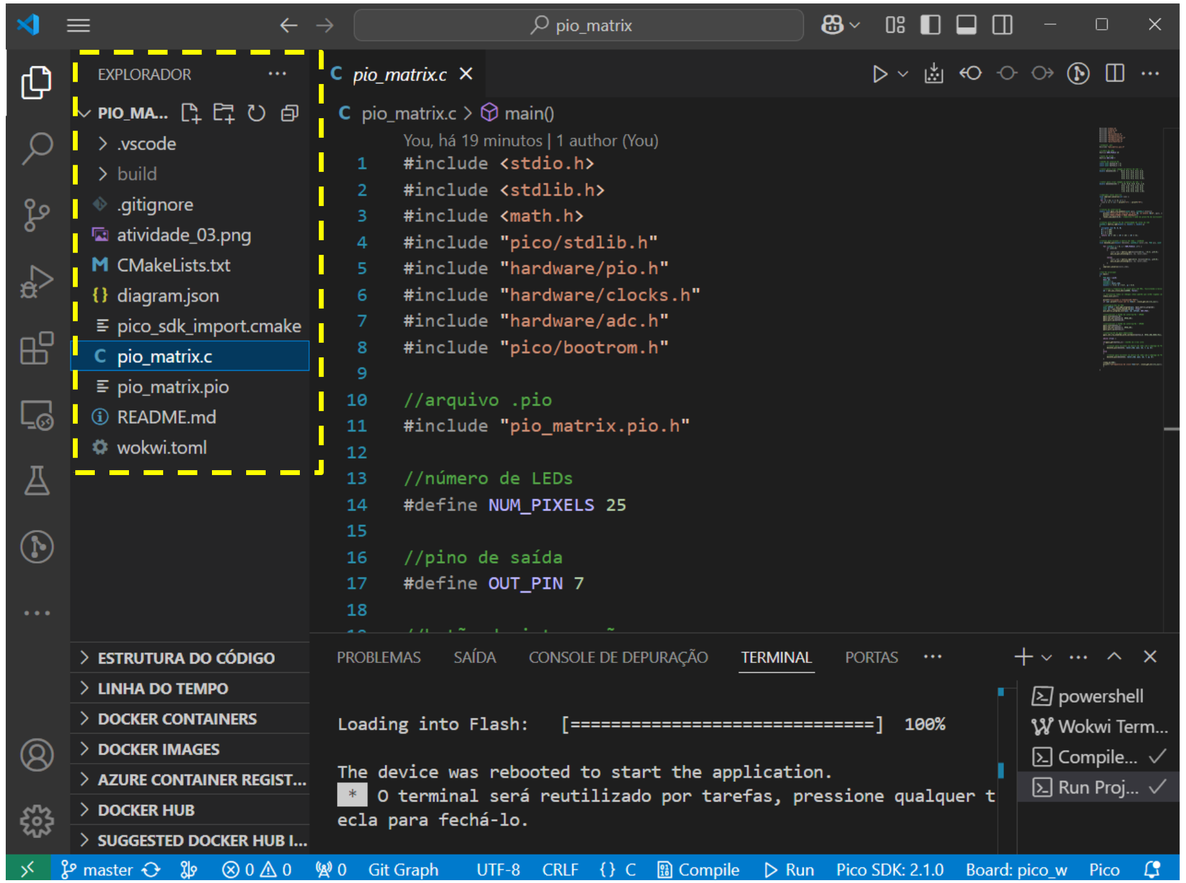


LED-RGB 5x5 WS2812



Prática 3 – Matrix de Leds (WS2812B)

VS Code:



$$f_{PIO} = \frac{128 \times 10^6}{16} = 8MHz$$

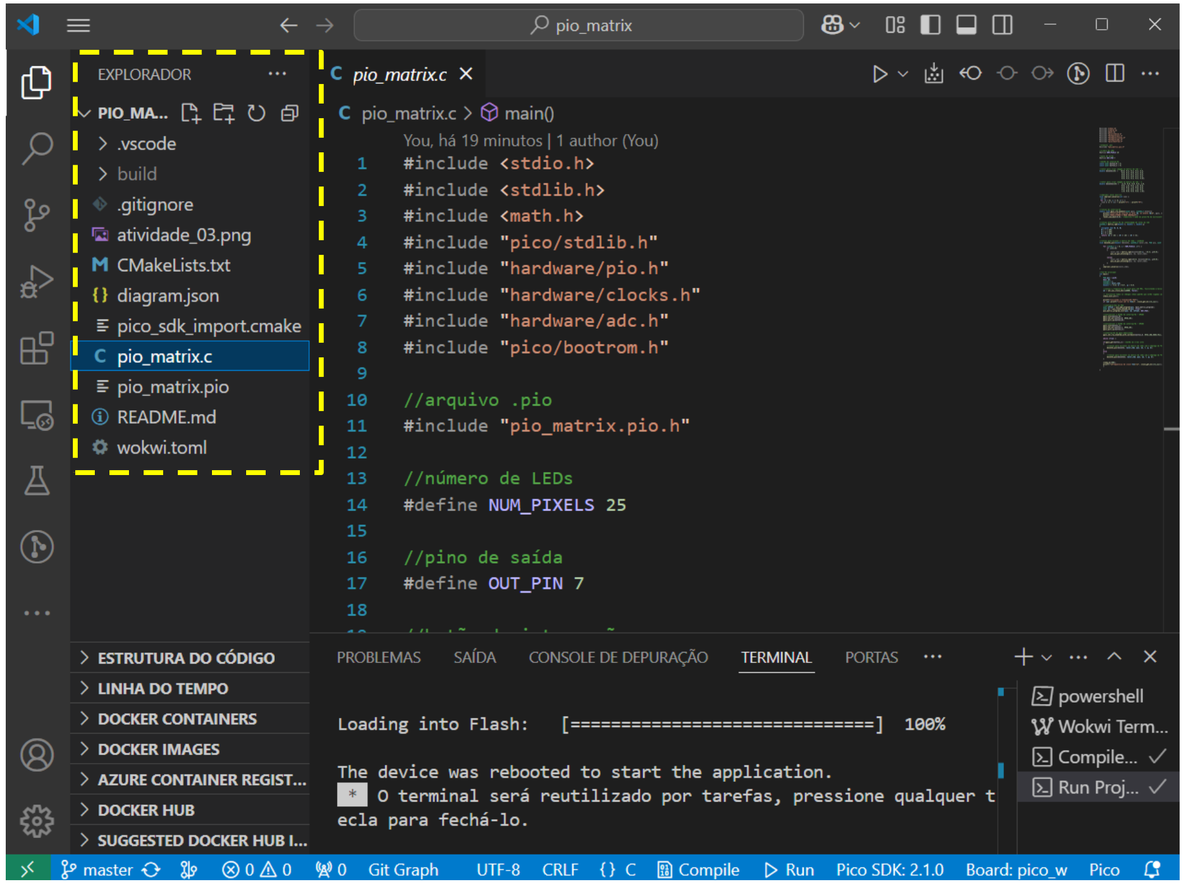
$$T_{PIO} = \frac{1}{f_{PIO}} = 125ns = 0{,}125\mu s$$

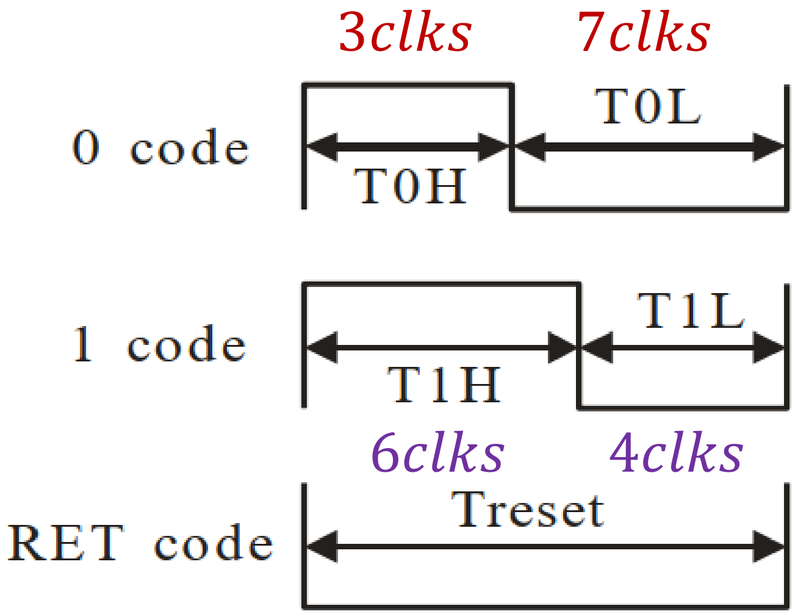
- Clock do PIO = 8MHz, 10x maior que a frequência de transferência de bits para o WS2812B (800KHz).

$$T_{WS} = \frac{1}{f_{PIO}} = 1{,}25\mu s$$

Prática 3 – Matrix de Leds (WS2812B)

VS Code:

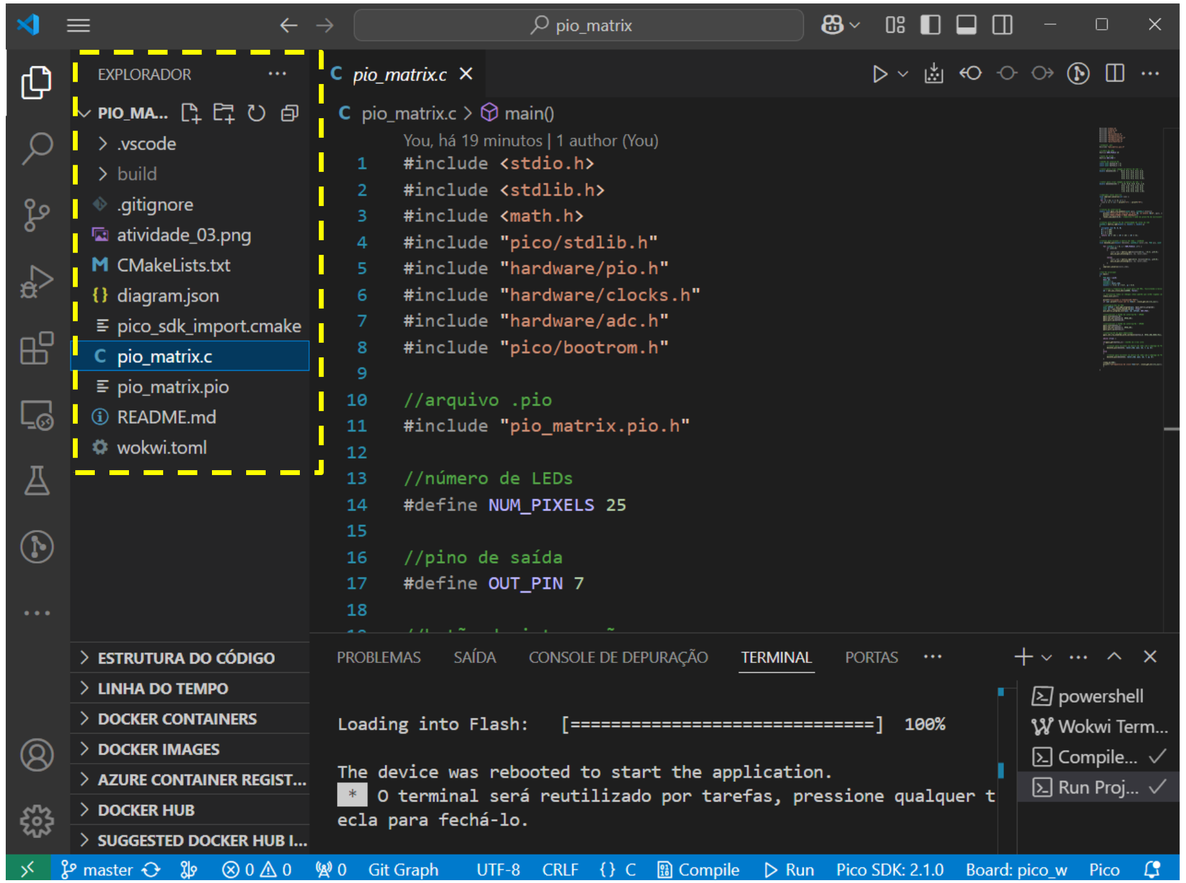




Lembrete: $10clks = 1{,}25\mu s$

Prática 3 – Matrix de Leds (WS2812B)

VS Code:



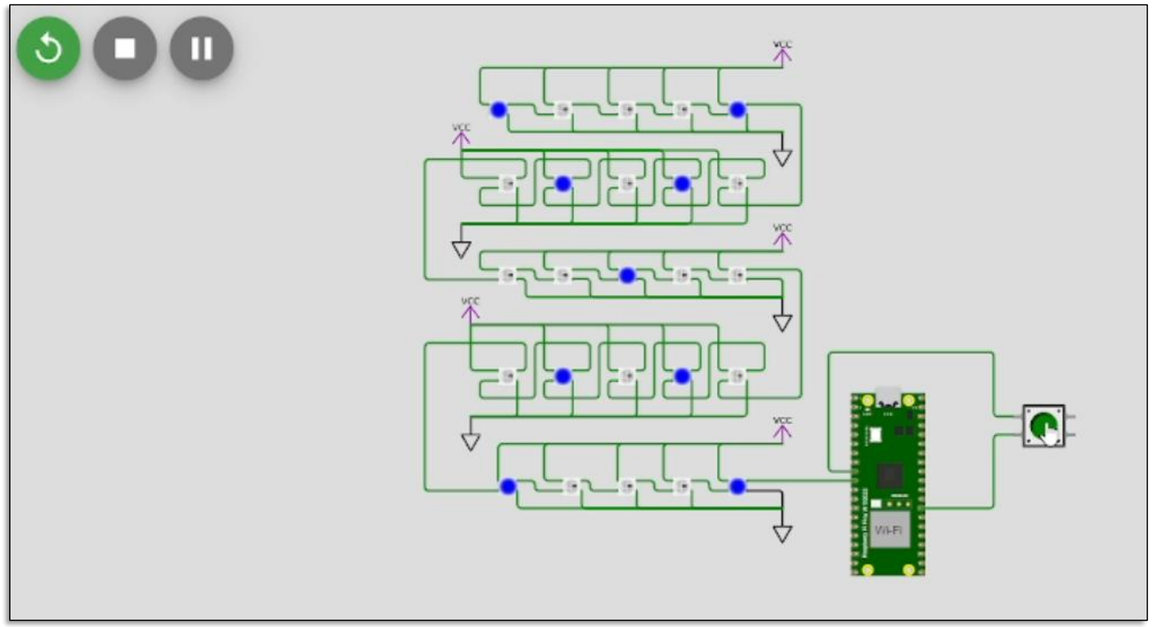
Data transfer time(TH+TL=1.25µs±600ns)

T0H	0 code ,high voltage time	0.4us	$0{,}375\mu s$
T1H	1 code ,high voltage time	0.8us	$0{,}775\mu s$
T0L	0 code , low voltage time	0.85us	$0{,}875\mu s$
T1L	1 code ,low voltage time	0.45us	$0{,}500\mu s$
RES	low voltage time	Above 50µs	

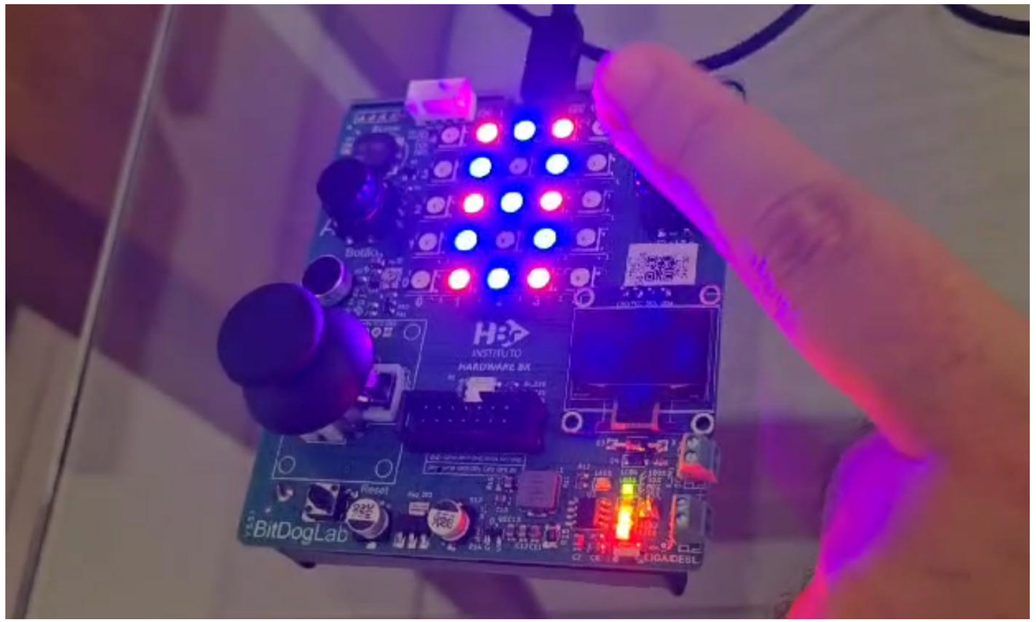
Vamos para o código!

Prática 3 – Matrix de Leds (WS2812B)

Simulação Wokwi:



Hardware BitDogLab:



Conclusões

Conclusões

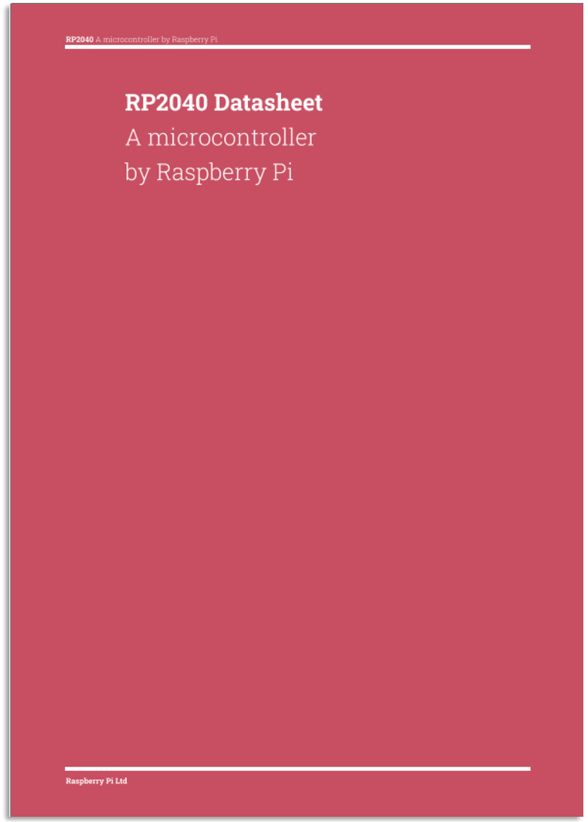
- O microcontrolador RP2040 conta com funções dedicadas para diferentes propósitos – a exemplo de E/S digitais (**GPIOs**), máquina de estado (PIOs), dentre outras .

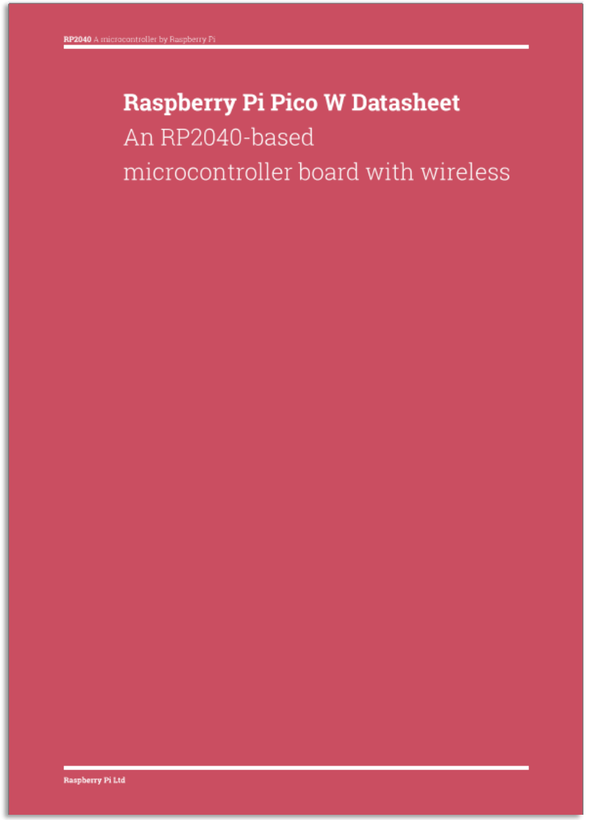
- O RP2040 possui rotinas de atraso (*delay*) que podem ser programadas em alto e baixo nível.

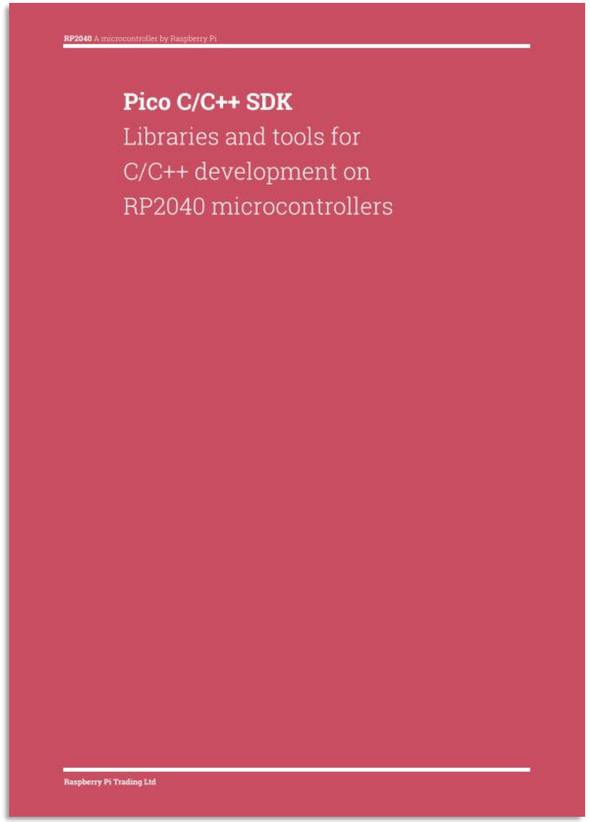
- O Kit de Desenvolvimento de *Software* (**SDK**) fornece um ambiente de programação que viabiliza a criação de uma ampla gama de rotinas de temporização.

- A ferramenta educacional **BitDogLab** e o simulador de eletrônica **Wokwi** podem ser empregados para realização de experimentos em periféricos externos presentes sistemas embarcados, a exemplo dos **LEDs WS2812B**.

Bibliografia Recomendada







embarca tech | Residência Tecnológica em Sistemas Embarcados

Microcontroladores – GPIO Parte 2

Unidade 4 | Capítulo 3 – Aula Síncrona (10/01/2025)

Executores: INSTITUTO FEDERAL Piauí | INSTITUTO FEDERAL Rio Grande do Norte | INSTITUTO FEDERAL Maranhão | INSTITUTO FEDERAL Ceará | cepedi | INSTITUTO HARDWARE BR

Coordenação: Softex

Iniciativa: MINISTÉRIO DA CIÊNCIA, TECNOLOGIA E INOVAÇÃO | GOVERNO FEDERAL BRASIL UNIÃO E RECONSTRUÇÃO